

邓懿轩·面试押题

A. 八股 / 基础原理题 共 77 题

方向一：AI Agent / 大模型 / RAG / 应用框架 (A1, 共 34 题)

【A1-01】【L1】Token、Embedding 与上下文窗口

题目：什么是 Token? Embedding 在 RAG 里起什么作用? 上下文窗口超长时你会怎么处理?

思考方向：从"文本如何变成模型可计算的数字"切入，再落到工程上的截断/压缩。

考察点：LLM 基础认知、向量语义表示、长上下文工程。

参考答案

- Token 是模型的最小处理单元，中文 1 字 $\approx$ 1~2 token；计费、限流、上下文长度都以 token 计。
- Embedding 把文本映射成高维稠密向量，语义相近的文本向量距离近，是向量检索/语义召回的基础。
- 超长上下文：滑动窗口截断（保留 system + 最近 N 轮 + 相关检索片段）、对话摘要压缩、把历史放到向量记忆里按需召回，而非全塞进 prompt。

评分要点

- **优秀：**能说出截断会丢信息、需要"分层记忆 + 摘要"策略，并关联 RAG 的检索片段注入。
- **及格：**讲清 Token/Embedding 概念与基本截断方式。
- **不足：**把 Token 等同于"字"，或不知道 Embedding 是向量。

常见追问：1) 为什么 1 个汉字常算 1~2 个 token? 2) 向量维度越高越好吗?

【A1-02】【L2】Transformer 自注意力机制

题目：简述 Transformer 的自注意力 (Self-Attention) 与多头注意力，为什么需要位置编码?

思考方向： $Q/K/V \rightarrow$ 相似度加权求和 $\rightarrow$ 多头并行捕捉不同子空间关系。

考察点：大模型底层架构理解。

参考答案

- Self-Attention: 输入经线性变换得 Q、K、V，用 $Q \cdot K^T$ 算词与词的相似度，softmax 归一化后加权求和 V，得到融合上下文的表示；除以 $\sqrt{d}$ 防止梯度消失。
- 多头：把 Q/K/V 拆成多组并行计算，不同头关注不同语义子空间（语法、指代、位置等），最后拼接。
- 注意力本身无视顺序，需位置编码（正弦/可学习/RoPE 旋转位置编码）注入位置信息；RoPE 是现代主流。

评分要点

- **优秀：**写出 Q/K/V 计算流程、 $\sqrt{d}$ 缩放原因、多头意义、RoPE 优于绝对位置编码（外推性）。
- **及格：**讲清 Q/K/V 与加权求和、多头和多位置编码作用。
- **不足：**只会背"注意力就是加权求和"，讲不出 Q/K/V 与缩放。

常见追问：1) 为什么自注意力复杂度是 $O(n^2)$ ，工程上如何优化? 2) RoPE 相对正弦编码的优势?

【A1-03】【L2】微调 vs RAG vs Prompt 怎么选

题目：业务上要提升模型效果，Prompt 工程、RAG、微调三者你怎么选？

思考方向：先按"知识是外部事实还是模型行为/风格"分类，再看成本与时效。

考察点：技术选型决策、成本意识。

参考答案

- 知识类需求（私域事实、实时数据、可溯源）→ RAG：不改模型、可热更新、可溯源、成本低。
- 行为/风格/格式类需求（固定话术、JSON 结构化输出、领域语气）→ Prompt（Few-shot/CoT）优先，成本最低。
- 微调（SFT/LoRA）：当需要固化复杂领域能力、压缩推理成本（用小模型替代大模型）、或 RAG+Prompt 都达不到效果时使用；缺点是数据/算力成本高、知识更新要重训。
- 实践常"组合拳"：微调让模型"会说"，RAG 让模型"知道"。

评分要点

- **优秀：**给出决策树并按知识/行为分类，指出可组合、以及微调小模型降本的价值。
- **及格：**讲清三者定位与 RAG 优先的原因。
- **不足：**上来就说"微调效果最好"，无成本/时效考量。

常见追问：1) 公司有大量客服话术，你会微调还是做 RAG？2) LoRA 和全参微调的区别与取舍？

【A1-04】【L3】幻觉成因与抑制

题目：大模型为什么会幻觉？在你做的问答 Agent 里，你是怎么抑制幻觉的？

思考方向：从"概率生成 + 知识缺失 + prompt 诱导"三层归因，再对应到你的工程手段。

考察点：幻觉机理、RAG 兜底与拒答策略。

参考答案

- 成因：① 本质是"预测下一个 token 的概率模型"，无事实校验；② 训练知识截止/缺失私域知识；③ prompt 未约束、上下文噪声干扰；④ 采样温度过高。
- 抑制：**检索侧**提升召回质量（混合召回+RRF+Rerank）、给出相关度阈值；**生成侧**强约束 prompt（"仅依据给定资料作答，不得编造，无依据则回答不知道"）、降低 temperature、要求引用来源；**兜底**低置信/无命中时拒答或转人工接管。
- 度量：用 RAGAS 的 Faithfulness（答案是否被上下文支撑）持续监控幻觉率。

评分要点

- **优秀：**三层归因 + 检索/生成/兜底三处手段 + 用 Faithfulness 量化，并说明拒答阈值怎么定。
- **及格：**能说出"无依据就拒答、prompt 约束、提升检索质量"。
- **不足：**只说"调 prompt 让它别乱说"。

常见追问：1) 拒答会不会误杀正常问题，怎么平衡召回率与拒答率？2) Faithfulness 怎么算的？

【A1-05】【L3】推理性能优化：KV Cache 与量化

题目：什么是 KV Cache？模型量化（INT8/INT4、GPTQ/AWQ）能带来什么收益与代价？

思考方向：从"自回归推理重复计算"解释 KV Cache，从"显存换精度"解释量化。

考察点：推理优化、成本与部署认知。

参考答案

- KV Cache：自回归生成时，已生成 token 的 K/V 结果不变，缓存后可避免每步重算整个序列，把生成阶段从 $O(n^2)$ 降到 $O(n)$ ，代价是显存占用随序列长度线性增长；配套 PagedAttention（vLLM）按页管理显存、Continuous Batching 提升吞吐。
- 量化：把权重/激活从 FP16 压到 INT8/INT4，显著降显存、提吞吐、可单卡部署更大模型；代价是精度损失，需评估效果回退。GPTQ 是训练后逐层量化，AWQ 保护重要通道精度更好。

评分要点

- **优秀**：讲清 KV Cache 为什么省算力、为什么费显存，能提到 PagedAttention/Continuous Batching、量化精度与显存的权衡。
- **及格**：能说 KV Cache 的作用与量化"省显存牺牲精度"。
- **不足**：没听过或答非所问。

常见追问：1) 首 token 延迟和吞吐如何权衡？2) 长上下文时 KV Cache 显存爆炸怎么办？

【A1-06】【L1】对话式 Agent 与 Function Calling 基础

题目：你怎么理解 Agent？Function Calling（工具调用）的工作原理是什么？

思考方向：Agent = LLM + 规划 + 工具 + 记忆；工具调用是"模型输出结构化调用意图 → 系统执行 → 结果回灌"。

考察点：Agent 基本概念、工具调用机制。

参考答案

- Agent：以大模型为决策核心，具备**意图理解/规划、工具调用、记忆、多轮交互**能力，能自主完成多步任务的系统。
- Function Calling：把工具的 name/description/参数 schema 以结构化形式告诉模型；模型判断需要调用时输出结构化的工具名+参数（而非自然语言）；系统执行后把结果作为新一轮上下文回灌模型，直到模型给出最终答案。
- 关键点：参数校验与异常兜底、工具描述质量决定模型能否选对工具、限制最大调用轮数防死循环。

评分要点

- **优秀**：讲清 schema 声明→模型决策→执行→回灌的完整闭环，并提到参数校验/防死循环。
- **及格**：讲清 Agent 由模型+工具+记忆构成、工具调用的基本流程。
- **不足**：把 Agent 等同于"套壳聊天"。

常见追问：1) 工具很多（几十个）时怎么让模型选准？2) 工具执行失败怎么让模型重试？

【A1-07】【L2】ReAct 推理框架原理

题目：ReAct 框架的核心思想是什么？它解决什么问题？

思考方向：Reason + Act 交替，让模型"边想边做"，用外部观察修正推理。

考察点：Agent 推理范式、工具编排。

参考答案

- ReAct = Reasoning + Acting：让模型循环执行 **Thought（思考）** → **Action（选择工具）** → **Observation（观察结果）**，直到得出 Final Answer。
- 解决的问题：纯推理（CoT）容易幻觉、无法获取外部信息；纯执行无法规划。ReAct 通过"思考过程显式化 + 工具反馈闭环"提升复杂任务的准确性与可解释性。
- 工程要点：Thought 可观测便于调试、限制最大步数、工具返回结果要做截断与结构化。

评分要点

- **优秀**：画出 Thought/Action/Observation 循环，说明与 CoT 的区别、以及调试与步数控制。
- **及格**：讲清"边想边做、用工具结果修正"。
- **不足**：只答"就是调用工具"。

常见追问：1) ReAct 容易陷入循环，你怎么防？2) ReAct 和 Function Calling 的关系？

【A1-08】【L2】Plan-and-Execute 与 ReAct 对比

题目：Plan-and-Execute 与 ReAct 有什么区别？你项目里什么时候用哪个？

思考方向：先全局规划再逐步执行 vs 每步即时决策。

考察点：Agent 范式选型。

参考答案

- ReAct: 每步即时"想一步做一步", 灵活、适合步骤少、环境反馈即时的任务; 缺点是缺少全局规划, 长任务易跑偏、token 消耗多。
- Plan-and-Execute: 先由 Planner 产出多步计划(任务列表), 再由 Executor 逐步执行, 可按需重规划(Re-plan); 适合步骤多、可分解的流程型任务; 缺点是计划一旦偏差需重规划、交互延迟更高。
- 选型: 知识问答/探索式取数用 ReAct; 多步业务流程自动化(如先查订单→再查库存→再生成报告)用 Plan-and-Execute。实践中常"规划+执行+反思"混合。

评分要点

- **优秀**: 讲清两者优缺点与重规划(Re-plan)机制, 并给出结合业务的选型判据。
- **及格**: 讲清"先规划后执行 vs 边想边做"的区别。
- **不足**: 把两者混为一谈。

常见追问: 1) 计划执行到一半发现前置条件不满足怎么办? 2) 怎么评估规划质量?

【A1-09】【L3】多 Agent 编排模式 (AgentScope)

题目: 多 Agent 有哪些常见协作模式? 你在 AgentScope 里怎么做多 Agent 编排? 什么时候不该用多 Agent?

思考方向: Supervisor (主管调度) / Swarm (平级移交) / 流水线 (Pipeline) 三种范式。

考察点: 多 Agent 架构、复杂度治理。

参考答案

- 常见模式: ① **Supervisor/主管**——中心 Agent 拆解任务并分发给子 Agent, 汇总结果(可控、易调试); ② **Swarm/平级移交**——Agent 之间按需转交控制权; ③ **流水线**——按固定顺序串联(如检索 Agent→分析 Agent→写作 Agent)。
- AgentScope 实践: 用统一的 message 传递机制 + MsgHub 广播共享上下文, 各 Agent 挂不同模型/职责/工具, 主控 Agent 做路由与结果聚合。
- 不该用的场景: 任务简单、单 Agent + 工具即可时, 多 Agent 只会增加延迟、成本与不可控性; "为用而用"是常见反模式。

评分要点

- **优秀**: 讲清三种模式差异与适用场景, 说明共享上下文/消息传递机制, 并给出"不该用"的判断。
- **及格**: 能说出主管/流水线模式与多 Agent 解决的问题。
- **不足**: 只知道"多个 Agent 一起干活", 无协作机制。

常见追问: 1) 多 Agent 之间上下文怎么共享与隔离? 2) 出错时如何定位是哪个 Agent 的问题?

【A1-10】【L2】Agent 持久化记忆机制

题目: Agent 的记忆怎么设计? 短期记忆与长期记忆分别怎么存?

思考方向: 短期=会话上下文, 长期=向量记忆/结构化画像。

考察点: 记忆分层设计、与 RAG 的结合。

参考答案

- 短期记忆: 当前会话多轮上下文, 存 Redis (带 TTL、按 sessionId), 超长时做摘要压缩或滑动窗口。
- 长期记忆: 把重要事实/用户偏好写入向量库(语义可召回)或关系库(结构化画像), 下次会话按 query 召回注入, 实现"跨会话记住用户"。
- 要点: 写入时机(哪些该记)、去重与更新(同一事实变更要覆盖)、隐私与权限(用户数据隔离)、记忆检索相关性阈值。

评分要点

- **优秀**：讲清短期/长期的分层与存储选型，提到写入策略、去重更新、隐私隔离。
- **及格**：讲清短期用会话上下文、长期用向量库。
- **不足**：只会说"把历史都塞进 prompt"。

常见追问：1) 记忆越来越多检索不准怎么办？2) 多用户共享一个 Agent 时怎么隔离记忆？

【A1-11】【L3】低置信兜底与人工接管设计

题目：你项目里"低置信/敏感问题转人工接管"是怎么实现的？置信度如何量化？

思考方向：从"何时该转人工"的信号定义，到状态机与工单流转。

考察点：Agent 兜底策略、人机协同。

参考答案

- 触发信号（多信号融合）：① 检索最高相关度低于阈值（召回不足）；② Rerank 分数低；③ 模型自评置信度低或明确表达不确定；④ 命中敏感词/合规词表；⑤ 用户显式要求转人工或连续多轮未解决。
- 实现：会话状态机（bot → pending_human → human → bot），转人工时冻结上下文并生成工单/事件通知坐席（WebSocket/消息队列），人工回复可回写并沉淀为知识，形成闭环。
- 要点：阈值需用线上 Badcase 校准（平衡误转与漏转），并统计转人工率、人工解决率作为迭代指标。

评分要点

- **优秀**：多信号触发 + 状态机 + 闭环沉淀，并强调阈值校准与指标监控。
- **及格**：能说出"检索分低/敏感词触发转人工 + 状态流转"。
- **不足**：只说"加个转人工按钮"。

常见追问：1) 转人工率过高会怎样，怎么优化？2) 人工接管期间用户又发消息怎么办？

【A1-12】【L2】Agent 效果评估指标

题目：Agent（区别于纯 RAG）的效果怎么评估？任务成功率和工具调用准确率怎么定义？

思考方向：拆成"最终结果、过程正确性、效率与体验"三层。

考察点：Agent 评估体系。

参考答案

- 结果层：任务成功率（端到端达成用户目标的比例）、答案正确率（与标准答案比对或人工标注）。
- 过程层：工具调用准确率（选对工具的比例）、参数正确率、平均步数/调用轮数（越低越好）、无效循环率。
- 体验/成本层：首 token 延迟、端到端时延、转人工率、单次对话成本（token 消耗）。
- 方法：构建评测集（含 golden 工具调用序列）+ 离线回放 + 线上 A/B 与用户反馈（点赞/点踩）。

评分要点

- **优秀**：区分结果/过程/成本三层，提到 golden 轨迹比对与 A/B。
- **及格**：能说清任务成功率与工具调用准确率两个核心指标。
- **不足**：只会说"看效果好不好"。

常见追问：1) 没有标注数据怎么快速评估？2) 用大模型当裁判（LLM-as-Judge）可靠吗？

【A1-13】【L1】RAG 全链路

题目：完整讲一下 RAG 的流程，每一步的作用是什么？

思考方向：离线入库链路 + 在线检索生成链路两条线。

考察点：RAG 全局认知（必答题）。

参考答案

- **离线**：文档预处理（解析/清洗）→ 分块（chunk）→ 元数据标注 → Embedding 向量化 → 写入向量库（Milvus）。
- **在线**：用户 query → （改写/扩写）→ 混合召回（向量 + BM25）→ 融合（RRF）→ Rerank 精排 → TopK 上下文拼装 prompt → 大模型生成 → 输出/引用/拒答。
- 质量闭环：RAGAS 评测（召回率、忠实度）+ Badcase 归因，反哺分块与召回参数迭代。

评分要点

- **优秀**：两条链路完整无遗漏，能指出每一步的常见坑（分块过碎、召回不准、上下文噪声）。
- **及格**：能串起"分块→向量化→检索→重排→生成"。
- **不足**：漏掉分块或重排，只说"向量库检索后喂给模型"。

常见追问：1) 哪一步对最终效果影响最大？2) 文档更新后怎么增量同步？

【A1-14】【L2】分块策略：语义分块与父子文档

题目：你项目用了"语义分块 + 父子文档"，它们分别解决什么问题？分块粒度怎么定？

思考方向：分块粒度是"检索精度 vs 上下文完整性"的权衡。

考察点：RAG 分块策略。

参考答案

- 固定长度分块简单但会在句子/语义中间切断；**语义分块**按语义边界（段落、标题、句子相似度突变处）切，保证块内语义完整。
- **父子文档**：用小的子块做向量检索（更精准），命中后返回其所属的父块/整段作为上下文（更完整），兼顾"检索准"与"信息全"。
- 粒度：chunk 太小语义不全、太大引入噪声且稀释向量；常见 300~800 token、overlap 10%~20%；最终靠 A/B 实验用指标（Recall@K、Faithfulness）定参数。

评分要点

- **优秀**：讲清父子文档"小子块检索、父块喂模型"的机制，并给出参数实验思路。
- **及格**：讲清语义分块的价值与 chunk size/overlap 的权衡。
- **不足**：只知道"按固定字数切"。

常见追问：1) overlap 太大会有什么问题？2) 表格/扫描件 PDF 怎么处理？

【A1-15】【L2】Query Rewrite 与 HyDE

题目：为什么要做查询改写？Query Rewrite 和 HyDE 分别怎么工作？

思考方向：用户口语化 query 与文档表述不匹配，需"翻译"到检索友好空间。

考察点：召回优化手段。

参考答案

- Query Rewrite：用 LLM 把口语化/省略/多轮依赖的 query 改写成完整、含关键词的标准查询（如"这个多少钱"→"XX 商品 价格"）；也含多轮指代消解、同义词扩展、子问题拆解。
- HyDE（Hypothetical Document Embeddings）：先让 LLM 生成一段"假设的答案文档"，再对这段假设文档做向量化检索。因为假设答案与真实文档在语义空间更接近，可显著提升召回；代价是多一次 LLM 调用、有引入偏差的风险。
- 取舍：延迟敏感可只做轻量 Rewrite；对召回质量要求高再加 HyDE，并用实验验证增益。

评分要点

- **优秀**：讲清 HyDE 的"假设答案做检索"机制、收益与额外延迟/偏差风险。
- **及格**：能说清改写的目的与 HyDE 大体思路。
- **不足**：不知道 HyDE 是什么。

常见追问：1) 改写失败（改错）怎么兜底？2) 改写会不会破坏原始关键词导致 BM25 召回变差？

【A1-16】【L1】向量检索 vs BM25 关键词检索

题目：向量检索和 BM25 各有什么优缺点？为什么不能只用一种？

思考方向：语义泛化能力 vs 关键词精确匹配能力，互补。

考察点：检索基础、混合召回动机。

参考答案

- 向量检索：基于语义相似，能理解同义/近义表达，泛化好；但对专有名词、型号、代码、数字不敏感，且依赖 embedding 质量。
- BM25：基于词频-逆文档频率的精确关键词匹配，对型号/术语/代码符号准；但不懂同义与语义，词不匹配就召回不到。
- 两者互补 → 混合召回，既保证语义理解又保证关键实体精确命中。

评分要点

- **优秀：**能举"商品型号/健康术语"的例子说明纯向量的漏召回问题。
- **及格：**讲清语义 vs 关键词的差异与互补。
- **不足：**认为"向量检索全面优于关键词检索"。

常见追问：1) BM25 的 k1、b 参数的作用？2) 中文分词对 BM25 的影响？

【A1-17】【L2】混合召回与 RRF 融合

题目：你的混合召回结果怎么融合？RRF 的公式和参数含义是什么？为什么用 RRF 而不是加权求和？

思考方向：两类检索的分数量纲不同，按"排名"融合更鲁棒。

考察点：RRF 原理与调参。

参考答案

- RRF (Reciprocal Rank Fusion)：对每个文档按它在各路结果中的排名 r 打分 $\text{score} = \sum 1/(k + r)$ ， k 为平滑常数（常用 60），再按总分排序。
- 用 RRF 的原因：向量相似度与 BM25 分数量纲/分布完全不同，直接加权求和需要归一化且权重难调；RRF 只用排名，天然规避归一化问题，鲁棒、少调参。
- 调优： k 越大越"平权"（削弱头部优势）， k 越小越放大头名；两路权重可通过多次出现加权（如给向量路结果乘系数）或按业务做 A/B。融合后再交给 Rerank 精排。

评分要点

- **优秀：**写出 RRF 公式、解释 k 的作用，说清"规避量纲/归一化"这一本质原因。
- **及格：**说清按排名倒数融合、比加权求和更鲁棒。
- **不足：**只会说"两路结果合并去重"。

常见追问：1) 如果两路召回结果都很差，RRF 能救吗？2) 有没有比 RRF 更好的融合方式？

【A1-18】【L2】Rerank 重排序：cross-encoder vs bi-encoder

题目：Rerank 的作用是什么？cross-encoder 与 bi-encoder 的区别？为什么要"先召回后精排"？

思考方向：粗排求快（召回全）、精排求准（算得慢但准）。

考察点：排序架构、效果与延迟权衡。

参考答案

- bi-encoder (Embedding 检索)：query 与 doc 分别编码成向量再算相似度，doc 向量可离线预计算，快但精度有限，适合大规模粗召回。
- cross-encoder (Rerank)：把 (query, doc) 拼在一起送进模型做交互式打分，精度高但无法预计算，只能对少量候选算，慢。

- "先召回后精排"：召回阶段用 bi-encoder 从百万级快速取 TopK（如 50~100），精排阶段用 cross-encoder 对这几十条精确打分取 TopN，兼顾速度与准确率。
- 调优：Rerank 模型选型（bge-reranker 等）、候选数与延迟的权衡、是否需要与 RRF 顺序配合。

评分要点

- **优秀**：讲清两种编码器的计算方式与延迟差异，说明召回/精排分层设计与 Rerank 模型选型。
- **及格**：能说清 Rerank 提升相关性、cross-encoder 更准更慢。
- **不足**：把 Rerank 与向量检索混为一谈。

常见追问：1) Rerank 让延迟涨了 200ms，值得吗，怎么权衡？2) 多路召回结果需要先融合再重排吗？

【A1-19】【L3】HNSW 向量索引原理与参数调优

题目：Milvus 常用的 HNSW 索引原理是什么？M、efConstruction、efSearch 三个参数怎么调？

思考方向：多层"跳表式"图，上层稀疏跳得远、下层稠密找得准。

考察点：向量索引底层、召回率与延迟调优。

参考答案

- HNSW（分层可导航小世界图）：构建多层邻近图，检索从上层的稀疏图快速跳跃逼近目标区域，再逐层下沉到最底层精细搜索，用少量计算换高召回。
- **M**：每个节点的最大连接数，越大召回越高、内存与构建越重（常用 16~32）。
- **efConstruction**：建索引时的候选队列大小，越大索引质量越好、构建越慢（常用 100~500）。
- **efSearch**：查询时的候选队列大小，越大召回越高、延迟越大，且**可在线调节**，是效果/延迟的主要旋钮。
- 权衡：HNSW 内存占用大、写入更新代价高；大规模+高更新场景可考虑 IVF 系列或 DiskANN，调参靠召回率-延迟曲线。

评分要点

- **优秀**：讲清分层图思想与三参数各自的"召回/内存/延迟"影响，能说出 efSearch 可在线调。
- **及格**：知道 HNSW 是图索引、M/ef 越大越准越慢。
- **不足**：只知道"用了 HNSW，效果不错"。

常见追问：1) 数据频繁更新会怎样，怎么处理？2) 内存吃紧有什么替代方案？

【A1-20】【L2】Milvus / FAISS / PGVector 选型

题目：你项目里同时提到 Milvus、FAISS、PGVector，为什么？三者的定位和选型标准是什么？

思考方向：从"数据规模、部署成本、运维能力、是否要事务/过滤"多维度比较。

考察点：向量存储选型。

参考答案

- FAISS：Facebook 的**库**，非服务，性能极强、可嵌入进程做本地/离线检索加速；缺点是分布式、持久化、权限、在线更新都要自己实现，适合单机/离线/评测场景。
- Milvus：**分布式向量数据库**，支持亿级规模、多索引（HNSW/IVF...）、标量过滤、多租户、持久化与水平扩展，适合生产在线服务；运维成本相对高。
- PGVector：在 PostgreSQL 上的向量扩展，最大优势是**能和业务数据同库**、复用 PG 的事务/SQL/权限，适合向量量级不大、已有 PG 且想省一套中间件的场景；规模大后性能不如专用库。
- 本项目：Milvus 作为在线主库，FAISS 做本地加速/评测，PGVector 兼容小数据量或已用 PG 的团队，形成"按规模与运维能力分层"的选型。

评分要点

- **优秀**：指出库 vs 服务 vs 扩展的本质差异，并给出规模/事务/运维的决策依据。
- **及格**：大致讲清三者定位与各自适用场景。

- **不足**：把三者说成"都差不多，随使用"。

常见追问：1) PGVector 的 HNSW 与 Milvus 的差别？2) 什么时候该从 PGVector 迁到 Milvus？

【A1-21】【L3】知识库权限隔离与多租户

题目：企业内部问答 Agent，不同角色能看到不同资料，你怎么做知识库的权限隔离？

思考方向：检索前过滤（pre-filter）vs 检索后过滤（post-filter），以及元数据设计。

考察点：多租户隔离、检索安全。

参考答案

- **核心手段**：给每个 chunk 打**权限元数据**（tenant_id / dept / role / 密级），检索时按当前用户身份做过滤。
- **pre-filter**（先过滤再向量检索，推荐）：Milvus/PGVector 支持标量过滤与向量检索同时进行，从候选阶段就排除无权数据，安全且效率可控；**post-filter**（先检索后过滤）：可能因过滤掉大量结果导致召回不足，且存在越权数据先被模型看到的风险。
- **进阶**：按租户物理隔离（独立 collection/分区）隔离更彻底但有资源成本；敏感字段脱敏；对"是否存在该文档"本身也算泄露的情况，需控制拒答话术不泄露信息。

评分要点

- **优秀**：明确 pre-filter 优先并说明原因，提到元数据设计、物理隔离成本与拒答话术泄露风险。
- **及格**：能说出"用元数据做权限过滤 + 分租户隔离"。
- **不足**：只想到"前端隐藏入口"，安全理解不足。

常见追问：1) 过滤条件很复杂时检索性能怎么保证？2) 用户权限变更后索引怎么同步？

【A1-22】【L2】RAGAS 评测：Recall@K / MRR / Faithfulness

题目：RAGAS 里 Recall@K、MRR、Faithfulness 分别衡量什么？怎么计算？你的评测集怎么构建？

思考方向：指标分两层：检索质量（Recall@K、MRR）与生成质量（Faithfulness、Answer Relevancy）。

考察点：RAG 评测体系。

参考答案

- **Recall@K**：TopK 检索结果中是否包含标准答案所需的知识片段，衡量召回是否"漏"，是最该先优化的指标（检索不到，生成再好也没用）。
- **MRR**（平均倒数排名）：正确答案首次出现的排名的倒数的均值，衡量"正确答案排得够不够前"，1 为最好。
- **Faithfulness**（忠实度）：答案中的每个论断是否能被检索到的上下文支撑，衡量幻觉程度；由 LLM 抽论断 + 逐条核验计算。
- **评测集**：从业务真实问题抽样 + 人工标注 ground truth（标准答案 + 应命中的文档/片段），覆盖高频、长尾、边界（无答案）三类问题；定期回归。

评分要点

- **优秀**：讲清"先治召回再治生成"的优先级，能说明各指标计算方式与评测集覆盖设计。
- **及格**：能说清三个指标各衡量什么。
- **不足**：只知道"用 RAGAS 打分"，说不清指标含义。

常见追问：1) 没有 ground truth 怎么评估（无参考评测）？2) 评测集会不会被"调参调过拟合"？

【A1-23】【L3】分块参数实验与 Badcase 全链路归因

题目：线上出现"用户能搜到的答案却答错了"，你怎么定位是分块、检索还是生成的问题？分块参数怎么做实验？

思考方向：用"分段验证"法把链路拆开，逐段比对定位瓶颈。

考察点：问题定位方法论、数据驱动的迭代闭环。

参考答案

- 链路归因（逐段验证）：① 人工确认该知识是否已入库、**分块是否把答案切碎**（分块问题）；② 检查该 chunk 在 TopK 结果里是否出现、排名第几（召回问题）；③ 若 chunk 在上下文里但答案仍错 → **生成问题**（prompt 约束不足、上下文噪声、模型能力）。
- 分块实验：控制变量法，固定检索/生成参数，只改 chunk size（如 256/512/1024）与 overlap（0/10%/20%），用 Recall@K + MRR + Faithfulness 对比；同一份评测集跑多组配置，选 Pareto 最优（效果/延迟/成本）。
- 闭环：沉淀 Badcase 库 → 归因分类统计（分块 X%、召回 Y%、生成 Z%）→ 优先解决占比最高的瓶颈。

评分要点

- **优秀：**给出"分段验证 + 控制变量实验 + Badcase 分类归因"的完整方法论，能指导下一步动作。
- **及格：**能说出先看知识是否入库、再看检索是否命中、最后看生成。
- **不足：**上来就"换更大的模型试试"，无归因思路。

常见追问：1) 怎么区分是"知识缺失"还是"分块切碎"？2) 效果和延迟冲突时怎么取舍？

【A1-24】【L1】Spring AI / Spring AI Alibaba / LangChain4j 定位

题目：Spring AI、Spring AI Alibaba、LangChain4j 三者是什么？为什么 Java 团队做 AI 应用要选框架？

思考方向：用"Java 生态的 LangChain"来类比，再讲各自差异。

考察点：Java AI 框架生态认知。

参考答案

- 三者都是**Java 生态的大模型应用开发框架**，把模型调用、Prompt 模板、Embedding、向量库、工具调用、会话记忆统一抽象，避免手写 HTTP 调各家 API。
- **Spring AI：**Spring 官方出品，强调与 Spring Boot 无缝集成（ChatClient/EmbeddingClient、自动配置、Advisor 机制），适合已有 Spring 体系的项目。
- **Spring AI Alibaba：**阿里在 Spring AI 基础上的增强，对通义千问等国产模型、阿里云向量库、多 Agent（Graph 编排）适配更好，国内落地友好。
- **LangChain4j：**社区版"Java LangChain"，组件最全（AiServices、@Tool、ChatMemory、EmbeddingStore、RAG 模块），灵活度高、非 Spring 强绑定。

评分要点

- **优秀：**讲清三者关系（Spring AI 官方 / Alibaba 增强 / LangChain4j 社区），并说明各自强项与选型倾向。
- **及格：**知道都是 Java 侧大模型框架、大体能做什么。
- **不足：**分不清三者，或以为要写 Python 才能做 AI。

常见追问：1) 为什么不用直接 HTTP 调 API？2) 你项目用了 AgentScope，和上面三者什么关系？

【A1-25】【L2】LangChain4j 核心组件：@Tool 与 ChatMemory

题目：LangChain4j 里怎么定义一个工具？ChatMemory 是怎么工作的？多用户会话如何隔离？

思考方向：注解式工具声明 + 按 memoryId 隔离的会话记忆。

考察点：框架实操、会话状态管理。

参考答案

- 工具定义：用 `@Tool("描述")` 标注方法，参数字段用 `@P("参数说明")`，框架自动生成 JSON Schema 传给模型；描述写得越清晰，模型选工具的准确率越高。
- AIServices：用接口 + 注解声明式定义 AI 服务（模型、记忆、工具、RAG 检索器按需组合），框架生成代理实现。
- ChatMemory：默认 MessageWindowChatMemory 保留最近 N 条消息，通过 `memoryId` 区分不同会话；多用户/多会话必须按 **memoryId 隔离**，生产上把 `memoryId` 映射到 Redis 存储并设 TTL，避免内存泄漏与串话。
- 注意：ChatMemory 是"消息级"记忆，与向量长期记忆不同；上下文超长要做摘要或窗口裁剪。

评分要点

- **优秀**：讲清注解生成 schema 的机制、`memoryId` 隔离与 Redis 持久化、以及窗口/摘要的取舍。
- **及格**：能说出 `@Tool` 定义工具、ChatMemory 用 `memoryId` 区分会话。
- **不足**：没实际用过，只会背概念。

常见追问：1) 工具方法抛异常框架怎么处理？2) 多个工具的并发调用支持吗？

【A1-26】【L3】自定义 EmbeddingStore 与并发会话隔离

题目：如果要接入框架不支持的向量库（比如你自研的存储），怎么扩展？高并发下会话/检索上下文怎么隔离？

思考方向：面向接口实现 + 无共享状态设计。

考察点：框架扩展能力、并发设计。

参考答案

- 扩展：实现框架的 EmbeddingStore 接口（add / search / remove 等方法），把自定义库的写读映射到接口语义；选型上优先用框架已适配的 Milvus/PGVector，自研仅在必要时。
- 并发隔离：① 每个请求的上下文（检索结果、工具中间态）用**请求级局部变量/ThreadLocal 或显式参数传递**，禁止放共享单例字段；② 会话记忆按 `memoryId` 隔离，底层用 Redis 而非共享 Map；③ 模型客户端/向量库连接池化并做限流，避免线程池被慢调用占满；④ 线程池隔离：AI 调用与业务调用分池，防止相互拖垮。
- 坑：Spring Boot 默认单例 Bean 下若把"当前用户/上下文"存成员变量，必然串话——这是 AI 应用最典型的生产事故。

评分要点

- **优秀**：指出"单例 Bean 存请求态"这一高危坑，并给出请求级隔离 + 池化 + 分池的完整方案。
- **及格**：能说出按会话 ID 隔离上下文、用连接池。
- **不足**：没意识到并发下的状态共享问题。

常见追问：1) ThreadLocal 在线程池复用下会有什么问题？2) 怎么压测验证隔离是否生效？

【A1-27】【L2】MCP 模型上下文协议

题目：MCP 是什么？它解决什么问题？一次 MCP 工具调用的流程是怎样的？

思考方向：把 MCP 类比"AI 世界的 USB-C"，统一模型与外设/工具/数据源的接入标准。

考察点：MCP 协议理解。

参考答案

- MCP (Model Context Protocol)：一套开放协议，用统一标准把**工具、资源、提示模板**暴露给大模型应用，解决"N 个模型 × M 个工具"各自定制对接的 M×N 集成噩梦。
- 角色：Host (宿主应用/Agent) ↔ Client ↔ Server (提供 tools/resources/prompts 的独立服务)；传输可用 stdio (本地进程) 或 SSE/HTTP (远程)。
- 流程：客户端连接 Server 并拉取工具清单 → 把工具以 schema 形式交给模型 → 模型决策 → 客户端经 JSON-RPC 调用 Server 执行 → 结果回灌模型。

- 价值：工具/数据源一次开发、多端复用，可插拔，安全边界更清晰（Server 独立部署、可鉴权）。

评分要点

- **优秀**：讲清 Host/Client/Server 角色、JSON-RPC 调用流程与"一次开发多端复用"的价值。
- **及格**：知道 MCP 是统一工具/数据接入的协议，能说出大致流程。
- **不足**：只知道"MCP 是个新概念"。

常见追问：1) MCP 的安全风险有哪些（提示词注入/越权）？2) 本地 stdio 与远程 SSE 怎么选？

【A1-28】【L3】MCP 与 Function Calling 的区别

题目：MCP 和 Function Calling 是一回事吗？既然有 Function Calling，为什么还要 MCP？

思考方向：Function Calling 是"模型的调用能力"，MCP 是"工具的接入与分发标准"。

考察点：概念辨析、架构分层理解。

参考答案

- Function Calling：模型侧能力——模型输出结构化的"要调用哪个函数、参数是什么"，属于**推理与决策层**，每个应用各自把工具注册进 prompt。
- MCP：**接入与分发层**——定义工具怎么暴露、被发现、被调用（协议 + 独立 Server），与具体模型解耦。
- 关系：二者互补而非替代。MCP Server 把工具标准化提供出来，Host 再把 MCP 里的工具"转译"成 Function Calling 的 schema 给模型；模型依旧靠 Function Calling 做决策，只是工具的来源统一成了 MCP。
- 价值：工具从"每个应用内硬编码"变成"可插拔的公共服务"，复用与治理（鉴权、版本、审计）都更好。

评分要点

- **优秀**：明确"能力层 vs 协议层"的分工，并说清二者如何协同工作。
- **及格**：能说出 MCP 是标准协议、Function Calling 是模型调用能力。
- **不足**：认为 MCP 就是 Function Calling 换个名字。

常见追问：1) 已有内部工具平台，还有必要上 MCP 吗？2) MCP 会不会引入额外延迟？

【A1-29】【L2】Dify / Coze 平台与 workflow 编排

题目：Dify 和 Coze 有什么差异？什么场景用平台、什么场景用代码框架？

思考方向：从用户群、部署方式、可定制性、与自有系统集成度对比。

考察点：平台认知、选型能力。

参考答案

- **Dify**：偏企业/开发者，开源可私有化部署，强在 RAG 知识库、workflow 编排、Agent 模式、API 发布与可观测；数据可自持，适合有数据合规要求的企业。
- **Coze (扣子)**：偏应用/运营侧，低代码可视化、插件生态丰富、发布渠道多（飞书/抖音/微信等），上手快；深度定制与私有化能力相对弱。
- **选型**：**快速验证/运营同学自助搭建/标准化场景** → 平台（Dify/Coze）；**需要深度定制、与自有微服务深度集成、复杂 Agent 逻辑、性能与成本可控** → 代码框架（Spring AI Alibaba / LangChain4j / AgentScope）。
- **实践**：平台做 MVP 验证，跑通后再用代码重写核心链路，兼顾速度与可控性。

评分要点

- **优秀**：讲清两者定位差异、私有化/合规考量，并给出"平台验证 + 代码落地"的组合策略。
- **及格**：能说出 Dify 偏企业开发、Coze 偏低代码应用。
- **不足**：只听过名字，说不清差异。

常见追问：1) Dify 工作流和 LangGraph 的区别？2) 平台方案的数据安全如何保障？

【A1-30】【L3】Agent 框架选型决策

题目：Spring AI Alibaba、LangChain4j、AgentScope、LangGraph 你会怎么选？给一个决策依据。

思考方向：从团队技术栈、语言、编排复杂度、生态与运维四维度决策。

考察点：架构选型、技术判断力。

参考答案

- **团队栈**：Java 为主、已有 Spring 微服务体系 → Spring AI Alibaba / LangChain4j（复用 IoC、监控、鉴权）；Python 数据/AI 团队 → LangChain + LangGraph。
- **编排复杂度**：单 Agent + 工具 → LangChain4j AiServices / Spring AI ChatClient 足够；多 Agent、有状态长流程、需要人工介入 → AgentScope（Java 多 Agent）或 LangGraph（Python 图编排、Checkpointing）。
- **模型与生态**：深度用阿里云/通义 → Spring AI Alibaba；要多模型自由切换、社区组件丰富 → LangChain4j。
- **运维与团队能力**：框架越“重”（多 Agent/图）调试与运维成本越高，要评估团队能否 hold 住；避免为了技术先进而引入无法维护的复杂度。
- 本项目：Java 微服务背景 → Spring AI Alibaba/LangChain4j 做模型与检索层，AgentScope 做多步编排，形成统一接入层，与现有系统无缝集成。

评分要点

- **优秀**：给出多维度决策矩阵，强调“团队能力/运维成本”这一常被忽视的维度，并有具体落地组合。
- **及格**：能按语言栈和编排复杂度给出合理选择。
- **不足**：只说“LangChain 最火所以用它”。

常见追问：1) 同一团队混用 Java 与 Python 框架怎么治理？2) 框架升级（大版本破坏性变更）怎么降低风险？

【A1-31】【L1】SSE 流式输出原理

题目：大模型回答为什么要用 SSE 流式？SSE 和 WebSocket 有什么区别？

思考方向：从“首字延迟体验”切入，对比两种长连接技术。

考察点：流式交互基础。

参考答案

- **流式价值**：大模型逐 token 生成，若等全部生成完再返回，用户要等数秒甚至十几秒；SSE 边生成边推送，**首 token 几百毫秒就能看到**，体感快很多。
- **SSE**：基于 HTTP 的单向（服务端→客户端）推送，`text/event-stream` 格式，自带断线重连与事件 id，实现简单、天然兼容 HTTP 基础设施（网关/负载均衡）。
- **WebSocket**：全双工、双向通信，适合需要客户端频繁实时上行的场景（协同编辑、语音）；实现与运维更重。
- **选型**：**大模型问答这类“请求一次、服务端持续推”的场景用 SSE 足够**；需要双向实时交互才用 WebSocket。

评分要点

- **优秀**：讲清首 token 体验价值、SSE 单向/自带重连、以及“何时该用 WebSocket”的边界。
- **及格**：能说清 SSE 是服务端单向推送、用于流式输出。
- **不足**：把 SSE 与 WebSocket 混为一谈。

常见追问：1) SSE 经过 Nginx 要怎么配置（缓冲/超时）？2) 如何取消正在进行的流式生成？

【A1-32】【L3】SSE 高并发、断线重连与背压

题目：要支撑 1000+ 并发流式问答且首 token < 300ms，你会怎么设计？断线重连和背压怎么处理？

思考方向：从接入层、模型层、连接管理、降级四处着手。

考察点：高并发流式架构、容错设计。

参考答案

- **接入层：**网关限流（按用户/租户）+ 负载均衡；SSE 是长连接，注意连接数上限、Nginx 关闭 proxy_buffering、放大 read timeout、keepalive 配置。
- **模型层：**用推理服务（vLLM）做连续批处理提吞吐；多模型路由与配额控制；热点问题做语义缓存直接命中，绕过模型调用。
- **连接与状态：**每个流分配 streamId，生成任务与 HTTP 连接解耦（任务写消息队列/Redis Stream，客户端断线重连可按 event id 续传）；会话上下文存 Redis，重连恢复不丢。
- **背压与降级：**客户端消费慢时限制缓冲队列长度，超出则丢弃中间态或降级为"生成完再返回"；模型超时降级到小模型/规则；全链路超时与重试；监控首 token 延迟、连接数、断线率。

评分要点

- **优秀：**讲清"生成与连接解耦 + event id 续传 + 背压策略 + 多层降级"，并能指出 SSE 长连接的资源上限风险。
- **及格：**能说清网关限流、连接池/异步处理、超时降级。
- **不足：**只答"加机器"。

常见追问：1) 一个节点能撑多少 SSE 连接，瓶颈在哪？2) 用户中途关闭页面，怎么及时释放模型资源？

【A1-33】【L3】Text2SQL 实现与安全

题目：你的 Text2SQL 是怎么实现的？怎么保证生成的 SQL 正确、安全、不拖垮数据库？

思考方向：Schema 注入 + 口径知识 + 校验 + 只读沙箱。

考察点：Text2SQL 工程化与安全。

参考答案

- 实现：把**表结构 + 字段注释 + 业务口径（如"签收业绩"怎么算）**作为上下文注入 prompt，让模型生成 SQL；结合 Few-shot 示例提升准确率；多表关联复杂时可先做 Schema 检索（RAG），只注入相关表，降低 token 与出错率。
- 正确性：生成后用 SQL 解析器做语法校验；对结果做抽样比对；复杂问题拆解为多步（先选表→再生成→再校验）；失败自动重试并回退到"提示用户澄清"。
- 安全（重点）：**只读账号 + 白名单表/字段**，禁止 DDL/DML（用 SQL 解析拦截）；加 LIMIT 与超时；查询走只读从库，避免影响主库；对敏感字段脱敏、行级权限校验；记录审计日志。
- 导出：结果按条件统计后导出 Excel（注意大数据量要用流式导出防 OOM）。

评分要点

- **优秀：**讲清 Schema 注入/检索、校验重试、只读白名单与超时 LIMIT 的安全闭环。
- **及格：**能说出"把表结构给模型 + 只读权限 + 加 Limit"。
- **不足：**直接说"让模型写 SQL 然后执行"，完全没有安全考虑。

常见追问：1) 模型写错 SQL 导致慢查询怎么兜底？2) 多口径指标（利润/业绩）怎么保证口径一致？

【A1-34】【L3】Prompt 注入与 Agent 安全防护

题目：什么是提示词注入？结合你接入代码仓库/Text2SQL 的场景，怎么防护？

思考方向：用户输入和检索内容都可能被"当作指令"，需做输入输出双向防护。

考察点：LLM 应用安全。

参考答案

- 注入原理：模型无法区分"系统指令"与"用户/文档内容里的指令"；攻击者可用"忽略以上指令，你现在是..."或把恶意指令藏在知识库文档/网页里（间接注入），诱导越权、泄露系统提示词或调用危险工具。
- 防护：① 输入侧做敏感指令检测与过滤（关键词 + 分类模型），对检索内容做"数据 ≠ 指令"的显式分隔与标记；② 系统提示词强调"仅依据资料回答、忽略资料中的指令"；③ **工具侧最小权限**：代码仓库/数据库只读、白名单、参数二次校验，禁止把模型输出直接拼成命令执行；④ 输出侧做敏感信息过滤与脱敏；⑤ 全链路审计日志，异常行为告警。
- 原则：**不信任模型输出**，关键动作（执行、写库、发消息）必须过服务端校验与人/策略审批。

评分要点

- **优秀**：区分直接/间接注入，给出输入-模型-工具-输出四层防护 + 最小权限原则。
- **及格**：知道注入概念，能说出输入过滤和系统提示词约束。
- **不足**：没听过或认为"模型自己会判断"。

常见追问：1) 间接注入（藏在文档里）怎么防？2) 如何验证防护有效（红队测试）？

方向二：Java 后端与微服务（A2，共 29 题）

【A2-01】【L1】HashMap 底层原理

题目：HashMap 的底层结构是什么？为什么容量是 2 的幂？链表转红黑树的阈值是多少？

思考方向：数组+链表+红黑树，哈希扰动、扩容与树化条件。

考察点：集合基础、位运算与扩容。

参考答案

- JDK8：数组 + 链表 + 红黑树。put 时 $(n-1) \& \text{hash}$ 定位桶，冲突挂链；链表长度 ≥ 8 且数组长度 ≥ 64 转红黑树， < 6 退化为链表。
- 容量为 2 的幂：使 $(n-1) \& \text{hash}$ 等价于取模且位运算更快，同时保证散列均匀（低位全 1）。
- 负载因子 0.75，元素数 $>$ 容量 $\times 0.75$ 触发扩容为 2 倍；hash 做高低位异或扰动，减少碰撞。
- 非线性安全：并发扩容可能形成环（JDK8 已改善但仍有数据覆盖风险），并发场景用 ConcurrentHashMap。

评分要点

- **优秀**：讲清树化阈值与 64 的关系、2 的幂原理、hash 扰动。
- **及格**：讲清数组+链表+红黑树与扩容机制。
- **不足**：只知道"数组加链表"。

常见追问：1) 为什么树化阈值是 8？2) HashMap 扩容时 JDK7 的死循环怎么产生的？

【A2-02】【L2】线程池参数与执行流程

题目：线程池 7 大参数是什么？任务提交后的执行顺序？你项目里线程池怎么配的？

思考方向：核心线程→队列→最大线程→拒绝策略，结合项目 1s→300ms 的并行改造。

考察点：并发工具、参数调优、生产实践。

参考答案

- 参数：corePoolSize、maximumPoolSize、keepAliveTime、unit、workQueue、threadFactory、handler。
- 执行流程：核心线程未滿 → 建核心线程；已滿 → 入队列；队列滿 → 建非核心线程到 max；再滿 → 拒绝策略（Abort/CallerRuns/Discard/DiscardOldest）。
- 项目实践（订单详情 1s→300ms）：把串行的"客户/商品/发货"三次远程调用改为并行，用固定大小线程池 + Future 汇总；池按业务隔离并命名（便于排查），队列用有界队列，拒绝策略按业务选（读场景 CallerRuns 兜底）。

- 要点：IO 密集型线程数可较大（2N CPU~），CPU 密集型约 N+1；务必用有界队列防任务堆积 OOM；线程泄漏需监控活跃线程数与队列深度。

评分要点

- **优秀**：讲清执行顺序的每个分支、有界队列的必要性、IO/CPU 密集型的线程数估算、业务隔离。
- **及格**：能说清 7 参数与"核心→队列→max→拒绝"的顺序。
- **不足**：只会 Executors.newFixedThreadPool，不了解拒绝策略。

常见追问：1) 为什么阿里规约禁用 Executors 创建线程池？2) 线程池里的任务抛异常了会怎样？

【A2-03】【L2】synchronized 锁升级

题目：synchronized 的锁升级过程是怎样的？锁信息存在哪里？

思考方向：对象头 Mark Word 的状态流转：无锁→偏向→轻量→重量。

考察点：并发底层、JVM 对象模型。

参考答案

- 锁状态存于对象头 Mark Word（2 bit 锁标志位）。
- 升级路径：无锁 → **偏向锁**（同一线程反复进入，只记录线程 ID，无 CAS）→ **轻量级锁**（多线程交替，CAS 自旋抢锁，失败则自旋）→ **重量级锁**（竞争激烈，自旋失败后挂起，依赖 OS 互斥量，涉及用户态/内核态切换）。
- 锁只升不降（偏向锁在 JDK15 后默认关闭/废弃，因撤销成本高）；重量级锁通过 Monitor（ObjectMonitor）实现，含 wait/notify 机制。
- 优化：自适应自旋、锁消除、锁粗化。

评分要点

- **优秀**：讲清每级锁的适用场景与实现方式、Mark Word 存储、JDK15 后偏向锁废弃。
- **及格**：能说出"偏向→轻量→重量"的顺序与大体原理。
- **不足**：只知道"synchronized 是重量级锁"。

常见追问：1) 偏向锁为什么要撤销，撤销的时机？2) synchronized 和 ReentrantLock 怎么选？

【A2-04】【L2】volatile 与 JMM

题目：volatile 保证了什么？为什么不保证原子性？什么是 happens-before？

思考方向：可见性 + 禁止重排序，但不含"复合操作的原子性"。

考察点：Java 内存模型。

参考答案

- volatile 保证**可见性**（写立即刷主存，读强制从主存取，底层是 lock 前缀指令 + 内存屏障）与**有序性**（禁止指令重排，如 DCL 单例需 volatile）。
- 不保证原子性：i++ 是"读-改-写"三步，多线程交叉执行仍会丢失更新，需用 synchronized/AtomicInteger（CAS）。
- JMM 定义主内存与工作内存抽象；happens-before 规则（程序顺序、监视器锁、volatile 变量、线程启动/终止、传递性）用于判断多线程下共享变量的可见性顺序。

评分要点

- **优秀**：讲清内存屏障、DCL 为什么需要 volatile、能列举 happens-before 规则。
- **及格**：讲清可见性/有序性、不含原子性。
- **不足**：答"volatile 就是线程安全"。

常见追问：1) DCL 单例不加 volatile 会出什么问题？2) volatile 数组能保证元素可见性吗？

【A2-05】【L2】AQS 原理

题目：AQS (AbstractQueuedSynchronizer) 的核心思想是什么？独占与共享模式有什么区别？

思考方向：state + CLH 双向队列 + CAS + LockSupport。

考察点：并发框架底层。

参考答案

- 核心：一个 volatile int **state** 表示同步状态 + 一个 CLH 双向等待队列 + CAS 修改 state + LockSupport.park/unpark 阻塞唤醒。
- 独占模式：state=1 表示已占用 (ReentrantLock, 可重入则 state+1)；共享模式：state 表示可用许可数 (Semaphore/CountDownLatch/ReadWriteLock 的读锁)。
- 加锁流程：CAS 抢 state 失败 → 包装成 Node 入队 → 自旋/阻塞等待 → 前驱释放后唤醒；释放时唤醒后继节点。
- 扩展：子类只需实现 tryAcquire/tryRelease (或共享版)，模板方法模式让 AQS 统一处理排队与阻塞。ReentrantLock 的公平锁即多了一层 hasQueuedPredecessors 判断。

评分要点

- **优秀：**讲清 state/队列/park 三要素、独占与共享差异、模板方法设计、公平锁实现差异。
- **及格：**能说出 state + CAS + 等待队列的核心机制。
- **不足：**只知道"AQS 是锁的底层"。

常见追问：1) 为什么用双向队列？2) 基于 AQS 手写一个不可重入锁的思路？

【A2-06】【L2】ConcurrentHashMap 并发安全实现

题目：JDK8 的 ConcurrentHashMap 如何保证并发安全？与 JDK7 有何区别？

思考方向：分段锁演进到 CAS + synchronized 锁桶头。

考察点：并发容器、锁粒度优化。

参考答案

- JDK7：Segment 分段锁 (继承 ReentrantLock)，默认 16 段，并发度 = 段数，锁粒度粗。
- JDK8：取消 Segment，Node 数组 + 链表/红黑树；**空桶用 CAS 写入，非空桶 synchronized 锁头节点**，锁粒度细化到桶；扩容时多线程协助迁移 (ForwardingNode 标识)。
- size：baseCount + CounterCell 数组分散热点，减少竞争。
- 注意：get 不加锁 (Node 的 val/next 用 volatile 保证可见性)；不允许 null 键值。

评分要点

- **优秀：**讲清 CAS 与 synchronized 的配合、多线程协助扩容、CounterCell 思想。
- **及格：**讲清 JDK7 分段锁 vs JDK8 桶级锁的差异。
- **不足：**只说"它是线程安全的"。

常见追问：1) size 在高并发下精确吗？2) 为什么不用 ReentrantLock 而用 synchronized？

【A2-07】【L1】JVM 内存结构

题目：JVM 运行时内存区域有哪些？哪些线程私有、哪些共享？

思考方向：私有 (程序计数器/虚拟机栈/本地方法栈) 与共享 (堆/方法区)。

考察点：JVM 基础。

参考答案

- 线程私有：程序计数器、虚拟机栈 (栈帧：局部变量表/操作数栈/动态链接/返回地址)、本地方法栈。
- 线程共享：堆 (对象实例，分新生代 Eden+Survivor、老年代)、方法区 (JDK8 起为元空间 Metaspace，存类元信息、常量、静态变量)。
- 常见异常：栈溢出 StackOverflowError (递归过深)、堆溢出 OOM: java heap space、元空间溢出 (类加载过多/动态代理/CGLIB)。

- 直接内存（NIO/Netty）不在堆内，也需关注。

评分要点

- **优秀**：区分 JDK7 永久代与 JDK8 元空间、说出各区域对应异常。
- **及格**：能列出五个区域并区分私有/共享。
- **不足**：把方法区等同于堆。

常见追问：1) 字符串常量池在哪？2) 元空间为什么替代永久代？

【A2-08】【L3】GC 调优与线上 OOM / CPU 飙高排查

题目：线上出现频繁 Full GC 或 OOM，你怎么排查？CPU 飙高怎么定位？

思考方向：从监控指标 → dump 分析 → 定位代码，形成标准排查动作。

考察点：故障排查实战能力。

参考答案

- Full GC 频繁：先用 jstat -gcutil 看各代使用率与 GC 频率；若老年代持续增长回收不掉 → 疑似内存泄漏，jmap dump 堆，用 MAT/VisualVM 分析支配树找大对象与引用链；若单次 GC 耗时高 → 堆过大或对象存活多，考虑分代比例/收集器（G1/ZGC）。
- OOM：结合 dump + 错误类型定位——heap space（对象过多/泄漏）、GC overhead limit（频繁 GC 无回收）、Metaspace（类加载泄漏）、unable to create native thread（线程数超限）。
- CPU 飙高：top -Hp 找高 CPU 线程 → 线程 id 转 16 进制 → jstack 查该线程栈，常见于死循环、正则回溯、频繁 GC、锁自旋。
- 工具：Arthas（dashboard/thread/trace/watch）做线上诊断，不改代码可定位慢方法与热点。

评分要点

- **优秀**：给出"指标→dump→引用链→代码"的完整路径，能区分几种 OOM 类型与 CPU 排查套路，提到 Arthas。
- **及格**：能说出 jstat/jmap/jstack + MAT 的基本用法。
- **不足**：只会说"重启一下"。

常见追问：1) 生产环境能直接 dump 吗，有什么风险？2) 怎么区分是内存泄漏还是内存不足？

【A2-09】【L1】Spring IoC 与 AOP

题目：什么是 IoC 和 AOP？AOP 的底层实现是什么？

思考方向：控制反转/依赖注入 + 面向切面，动态代理实现。

考察点：Spring 核心。

参考答案

- IoC：对象创建与依赖装配交给容器，通过构造器/字段/Setter 注入，降低耦合、便于测试与替换实现。
- AOP：把日志、事务、鉴权等横切关注点从业务代码中抽离，通过切点表达式在方法前后织入增强。
- 底层：JDK 动态代理（目标类实现接口，基于反射 InvocationHandler）与 CGLIB（无接口时，生成子类字节码增强）；Spring Boot 2.x 起默认 proxyTargetClass=true 倾向 CGLIB。
- 注意：AOP 生效依赖代理对象，**同类内部方法调用不走代理，切面/事务会失效。**

评分要点

- **优秀**：讲清两种代理的选择条件、内部调用失效的原因。
- **及格**：讲清 IoC/AOP 概念与动态代理实现。
- **不足**：只知道"Spring 帮我 new 对象"。

常见追问：1) JDK 代理和 CGLIB 的性能差异？2) AOP 的 5 种通知类型？

【A2-10】【L2】Bean 生命周期与循环依赖三级缓存

题目：Spring Bean 的生命周期？循环依赖是怎么用三级缓存解决的？为什么不能只靠二级？

思考方向：实例化→属性填充→初始化→销毁，缓存解决"提前暴露引用"。

考察点：Spring 源码机制。

参考答案

- 生命周期：实例化（构造）→ 属性填充（依赖注入）→ Aware 回调 → BeanPostProcessor.before → @PostConstruct/InitializingBean → AOP 代理生成 → after → 使用 → 销毁（@PreDestroy/DisposableBean）。
- 三级缓存：singletonObjects（成品）、earlySingletonObjects（半成品）、singletonFactories（ObjectFactory）。A 依赖 B、B 依赖 A 时，A 实例化后把"能产出 A 早期引用的工厂"放入三级缓存；B 注入时从三级缓存拿到 A 的早期引用（必要时提前生成代理）放入二级缓存，B 完成后再让 A 完成初始化。
- 为什么不能只两级：若只用二级直接存早期引用，则无法在"需要时"才把 AOP 代理提前暴露——三级缓存存的是工厂，保证**只有一个代理对象**，避免注入原始对象与最终代理不一致。
- 限制：只能解决**单例、字段/Setter 注入**的循环依赖；构造器注入与原型 Bean 无法解决（会抛异常）。

评分要点

- **优秀：**讲清三级缓存各自作用、工厂延迟生成代理的意义、构造器注入/原型失效的边界。
- **及格：**能说出三级缓存名字与大体解决思路。
- **不足：**只知道"Spring 用三级缓存解决循环依赖"。

常见追问：1) @Async 与循环依赖一起用会怎样？2) 为什么构造器循环依赖解决不了？

【A2-11】【L2】事务传播行为与 @Transactional 失效

题目：Spring 事务传播行为有哪些？@Transactional 在哪些场景会失效？

思考方向：7 种传播行为 + 代理失效的几类典型坑。

考察点：事务机制、生产踩坑经验。

参考答案

- 7 种传播：REQUIRED（默认，加入现有事务）、REQUIRES_NEW（挂起当前、新开事务）、SUPPORTS、NOT_SUPPORTED、MANDATORY、NEVER、NESTED（嵌套，保存点回滚）。
- 失效场景：① 同类内部方法自调用（不走代理，最常考）；② 方法非 public；③ 异常被 try-catch 吞掉未抛出；④ 抛的是受检异常默认不回滚（需 rollbackFor = Exception.class）；⑤ 多线程/异步调用（事务在别的线程）；⑥ 数据库引擎不支持事务（MyISAM）。
- 解决：注入自身代理 / AopContext.currentProxy() / 拆分到另一个 Bean；统一配 rollbackFor。

评分要点

- **优秀：**能完整列举失效场景并给出解决方案，讲清 REQUIRES_NEW 与 NESTED 的差别。
- **及格：**能说出 4~5 种传播行为与 2~3 类失效场景。
- **不足：**只会说"加个 @Transactional 就有事务了"。

常见追问：1) REQUIRES_NEW 的挂起是怎么实现的？2) 分布式场景下本地事务还够用吗？

【A2-12】【L2】MyBatis 原理与插件机制

题目：MyBatis 的执行流程是怎样的？#{ } 和 \${ } 的区别？插件（Interceptor）怎么工作？

思考方向：SQL 解析→参数映射→执行→结果映射，插件基于动态代理拦截四大对象。

考察点：ORM 框架原理。

参考答案

- 流程：加载配置/Mapper → 生成 SqlSession → 从 MappedStatement 取 SQL → 参数处理 (ParameterHandler) → 执行 (Executor→StatementHandler→JDBC) → 结果映射 (ResultSetHandler)。
- #{} 是预编译占位符 (PreparedStatement 参数, 防 SQL 注入) ; \${} 是字符串拼接 (有注入风险, 仅用于表名/排序等动态片段, 需白名单校验)。
- 插件：基于 JDK 动态代理, 可拦截 Executor / StatementHandler / ParameterHandler / ResultSetHandler 四大对象, 典型用途是分页 (如 PageHelper)、审计、多租户、SQL 日志; 注意多个插件会层层包装, 且拦截点选错会影响功能。

评分要点

- **优秀**：讲清四大对象与代理链、#{}/\${} 的安全差异与使用边界。
- **及格**：能说出执行流程与 #{} 预编译的区别。
- **不足**：不知道 MyBatis 内部组件。

常见追问：1) 分页插件原理是什么？2) 一级/二级缓存的坑？

【A2-13】【L2】Spring Boot 自动配置原理

题目：@SpringBootApplication 包含哪些注解？自动配置是怎么实现的？

思考方向：@EnableAutoConfiguration → spring.factories / AutoConfiguration.imports → 条件装配。

考察点：Spring Boot 核心机制。

参考答案

- @SpringBootApplication = @SpringBootConfiguration + @ComponentScan + @EnableAutoConfiguration。
- 自动配置：@EnableAutoConfiguration 通过 @Import(AutoConfigurationImportSelector) 读取 META-INF/spring.factories (Boot 2.7+/3.x 为 META-INF/spring/AutoConfiguration.imports) 中的候选配置类；再由 @Conditional 系列 (@ConditionalOnClass/@ConditionalOnMissingBean/@ConditionalOnProperty) 按需生效。
- 所以"引入 starter 就有默认配置、自己定义了 Bean 就覆盖默认", 本质是条件装配；用户配置优先于自动配置。
- 可自定义 starter：写自动配置类 + 条件注解 + spring.factories 注册, 实现"引依赖即用"。

评分要点

- **优秀**：讲清 import selector 加载机制 + 条件装配 + 覆盖优先级 + 自定义 starter 思路。
- **及格**：能说出三大注解与条件装配。
- **不足**：只会背注解名。

常见追问：1) 自动配置类的加载顺序怎么控制？2) 怎么排除某个自动配置？

【A2-14】【L1】设计模式实战

题目：你在项目里用过哪些设计模式？结合实际场景讲讲。

思考方向：不要背 23 种，挑 2~3 个真实用过的讲透。

考察点：工程思想与落地能力。

参考答案

- **策略模式**：支付方式 (微信/支付宝/藏宝岛分支) 用一个 PayStrategy 接口 + 多实现, 按类型路由, 避免大量 if-else; 新增支付方式只需加实现。
- **模板方法**：批量发货/异步任务定义统一的"校验→执行→记录失败→重试"骨架, 子类实现具体明细处理。
- **责任链**：AI 问答的输入预处理 (敏感词→改写→路由) + 后置处理 (引用标注→脱敏→落库) 串成链, 便于插拔。

- **单例/工厂/观察者**按需提及。关键是把"解决了什么扩展性问题"讲清楚，而不是罗列。

评分要点

- **优秀**：结合本项目真实场景（支付分支、批量任务、Agent 处理链），讲清"为什么用它、带来什么可扩展性"。
- **及格**：能说出 2~3 个模式并在业务场景中合理应用。
- **不足**：只背定义，说不出项目落点。

常见追问：1) 策略模式怎么根据类型选择实现（Spring 注入 Map）？ 2) 责任链和 AOP 的关系？

【A2-15】【L2】MySQL B+ 树索引与最左匹配

题目：为什么用 B+ 树而不用 B 树/哈希？什么是聚簇索引、回表、覆盖索引、最左前缀？

思考方向：从磁盘 IO 与范围查询的角度说明选型。

考察点：索引原理（面试高频）。

参考答案

- B+ 树：非叶子节点只存索引、不存数据，单节点能放更多键 → 树更矮（3~4 层可存千万级），磁盘 IO 少；叶子节点用链表相连，天然支持范围查询与排序。B 树数据分散在各层，范围查询要回溯；Hash 索引只支持等值、不支持范围与排序。
- InnoDB 主键为**聚簇索引**（叶子存整行数据）；二级索引叶子存主键值，查询非索引列需拿主键回表。
- **覆盖索引**：查询列全在索引里，无需回表；**最左前缀**：联合索引 (a,b,c) 必须从 a 开始连续匹配才能用上，跳过 a 直接用 b 失效。
- 索引下推（ICP）：在存储引擎层就用索引列做过滤，减少回表。

评分要点

- **优秀**：讲清树高与 IO 的关系、回表/覆盖索引、最左前缀的连续性要求与 ICP。
- **及格**：能说出 B+ 树优势与聚簇索引、回表概念。
- **不足**：只说"索引能加快查询"。

常见追问：1) 联合索引 (a,b,c)，where b=1 and c=2 会走索引吗？ 2) 为什么建议用自增主键？

【A2-16】【L2】MVCC 与事务隔离级别

题目：MySQL 的四种隔离级别？MVCC 是怎么实现的？它解决了什么问题？

思考方向：undo log 版本链 + ReadView 判断可见性。

考察点：事务与并发控制。

参考答案

- 隔离级别：读未提交、读已提交（RC）、可重复读（RR，InnoDB 默认）、串行化。分别解决脏读、不可重复读、幻读（RR 下通过间隙锁 + MVCC 基本解决幻读）。
- MVCC：每行有隐藏的 trx_id 与 roll_pointer；更新时旧版本写入 **undo log** 形成版本链；查询时生成 **ReadView**（记录当前活跃事务列表），沿版本链找到对当前事务可见的版本。
- RC 与 RR 的差异：RC 每次查询都生成新 ReadView（能看到别人已提交的最新数据）；RR 只在首次查询时生成一次 ReadView（整个事务看到同一快照）。
- MVCC 主要解决读写不阻塞（快照读），但当前读（select for update / update）仍需加锁。

评分要点

- **优秀**：讲清 ReadView 生成时机导致 RC/RR 差异、快照读 vs 当前读。
- **及格**：能说出四级别与 undo log 版本链 + ReadView 的基本机制。
- **不足**：分不清各级别解决的问题。

常见追问：1) RR 下幻读真的完全解决了吗？ 2) 间隙锁什么时候会加？

【A2-17】【L3】慢 SQL 优化与死锁排查

题目：一条 SQL 执行很慢，你怎么排查和优化？线上出现死锁怎么办？

思考方向：explain 看执行计划 → 索引/回表/数据量 → 改写 SQL；死锁看日志与加锁顺序。

考察点：SQL 优化实战。

参考答案

- 排查：开慢查询日志定位 SQL；**explain** 看 type（全表扫描 ALL / index / range / ref / const）、key 实际用到的索引、rows 扫描行数、Extra（Using filesort/temporary 需警惕）。
- 优化：建合适的联合索引（遵循最左前缀、高选择性列在前）、用覆盖索引避免回表、避免索引失效（函数运算/隐式类型转换/前导 %like/or 混用）、大分页用"延迟关联"或游标（where id > lastId）、减少 select *、拆大事务。
- 死锁：show engine innodb status 看 LATEST DETECTED DEADLOCK，分析两个事务的加锁顺序；常见成因是**不同事务以相反顺序加锁**（如先锁 A 再锁 B vs 先锁 B 再锁 A）或间隙锁范围冲突；解决靠统一加锁顺序、缩小事务范围、必要时降级隔离级别。
- 兜底：加超时（innodb_lock_wait_timeout）、重试机制、热点更新用乐观锁/队列串行化。

评分要点

- **优秀：**能读懂 explain 关键字段、给出多种索引失效场景与分页优化技巧，死锁能定位到加锁顺序。
- **及格：**能说出 explain + 建索引 + 避免索引失效。
- **不足：**只会说"加个索引"。

常见追问：1) 索引越多越好吗，代价是什么？2) 大表加索引会锁表吗？

【A2-18】【L1】缓存穿透、击穿、雪崩

题目：缓存穿透、击穿、雪崩分别是什么？怎么解决？

思考方向：三者成因不同：查不存在的数据、热点 key 失效、大批 key 同时失效。

考察点：缓存治理（必考题）。

参考答案

- **穿透：**查询 DB 中不存在的数据，缓存永远不命中，请求全打到 DB。方案：缓存空值（短 TTL）、布隆过滤器拦截、参数校验。
- **击穿：**某个热点 key 过期瞬间，大量并发请求同时回源。方案：互斥锁/分布式锁只放一个请求回源、逻辑过期（不设过期时间，异步刷新）、热点 key 永不过期。
- **雪崩：**大量 key 同一时刻过期或缓存服务宕机，DB 瞬间被打垮。方案：过期时间加随机抖动、多级缓存（本地 Caffeine + Redis）、缓存高可用（集群/哨兵）、限流降级兜底。
- 本项目对应：商品详情用"按商品 ID 加锁 + 二次检查"解决击穿，即同一商品只放一个请求回源。

评分要点

- **优秀：**三类成因与方案一一对应，能结合"按 ID 加锁 + DCL"与逻辑过期讲实现细节。
- **及格：**能说清三者区别与各自的解法。
- **不足：**三者混为一谈。

常见追问：1) 布隆过滤器有误判吗，怎么处理？2) 逻辑过期怎么保证数据一致性？

【A2-19】【L2】缓存与数据库一致性

题目：怎么保证缓存与数据库的一致性？为什么推荐"先更新 DB 再删缓存"？

思考方向：更新策略对比 + 延迟双删 + 最终一致（Canal 订阅 binlog）。

考察点：一致性设计。

参考答案

- 不推荐"更新缓存"：并发写会乱序，且很多写不值得缓存（白写）。
- **Cache Aside（先更新 DB，再删缓存）**：出现不一致的概率最低——若同时有读请求，读线程可能读到旧值写回缓存，但需要"删除 + 延迟双删"来缩小窗口。
- 强一致方案：读请求与写请求串行化（分布式锁，性能差）；**最终一致**：用 Canal 订阅 MySQL binlog，异步删除/重建缓存，解耦且可靠，是主流生产方案。
- 兜底：缓存设过期时间（兜底一致性的最后防线）；对一致性要求极高的读走主库。

评分要点

- **优秀**：讲清"为什么删而不是更新"、双删的必要性与残留窗口、Canal 最终一致方案。
- **及格**：能说出 Cache Aside 与延迟双删。
- **不足**：认为"先删缓存再更新 DB"就万事大吉。

常见追问：1) 延迟双删的延迟时间怎么定？2) 能不能做到强一致？

【A2-20】【L2】分布式锁与 Redisson 看门狗

题目：用 Redis 实现分布式锁要注意什么？Redisson 的看门狗机制是什么？

思考方向：SETNX+过期时间→原子性→续期→可重入→释放安全性。

考察点：分布式并发控制。

参考答案

- 基本实现：`SET key value NX PX 30000`，必须"加锁 + 过期"原子完成（旧版 SETNX + EXPIRE 两步会因宕机死锁）。
- 释放锁：value 用唯一标识（UUID+线程 ID），用 Lua 脚本"判断是自己的锁再删"，防止误删别人的锁。
- 看门狗（watchdog）：Redisson 加锁成功后台起一个定时任务，默认每 10s（租期 30s 的 1/3）检查并自动续期，避免业务未执行完锁就过期；业务宕机则续期停止，锁自然到期释放。也支持可重入（Hash 结构记录重入次数）与 RedLock 多节点。
- 局限：Redis 主从切换可能丢锁（异步复制），强一致场景需 RedLock 或 ZooKeeper/etcd。

评分要点

- **优秀**：讲清原子加锁、Lua 安全释放、看门狗续期与可重入实现，并指出主从切换丢锁风险。
- **及格**：能说出 SETNX+PX、唯一 value 释放、看门狗续期。
- **不足**：只会 SETNX 且不加过期时间。

常见追问：1) Redisson 看门狗在什么情况下会失效？2) 分布式锁和分布式事务能互相替代吗？

【A2-21】【L2】Redis 主从读写分离与高可用

题目：你项目做了 Redis 主从读写分离，怎么落地？主从延迟导致读到旧数据怎么办？

思考方向：写走主、读走从的收益与代价，延迟与一致性权衡。

考察点：缓存架构、一致性权衡。

参考答案

- 落地：写命令与失效操作走主节点，读请求分流到从节点，缓解主节点读压力、提升读吞吐；配合哨兵（Sentinel）或 Cluster 做故障自动切换。
- 主从延迟：Redis 复制是异步的，写主后立刻读从可能读到旧值（读到"回退"的数据）。方案：① 对一致性敏感的读（如刚下单后查订单）强制走主；② 关键写后加短暂等待或"读己之写"路由；③ 用版本号/延迟双删兜底；④ 监控主从复制 offset 差值。
- 注意：从节点故障、主从切换瞬间会有连接抖动，客户端需重试；集群模式下要处理跨 slot 的多 key 操作。

评分要点

- **优秀**：讲清读写分离的收益、异步复制的延迟本质，并给出"读己之写"的分流策略。

- **及格**：能说出写主读从、延迟导致不一致与基本兜底。
- **不足**：认为读写分离"完全没有副作用"。

常见追问：1) 哨兵和集群的区别？2) 主从全量同步和增量同步的触发条件？

【A2-22】【L3】大 key / 热 key 治理

题目：什么是大 key 和热 key？会引发什么问题？怎么发现和治理？

思考方向：大 key 是"单个 key 数据量过大"，热 key 是"单个 key 访问量过大"，都会拖垮单节点。

考察点：Redis 生产治理。

参考答案

- 大 key：价值如成员数万的 Hash/List、几 MB 的 String。危害：单次操作阻塞 Redis 单线程（其他请求排队）、网络带宽打满、删除时阻塞、集群分片不均。治理：拆分（按业务/哈希分桶）、压缩、惰性删除（unlink 异步删）、设置合理过期。
- 热 key：如秒杀商品、爆款详情。危害：流量集中到单个分片/节点，打满单节点。治理：**多级缓存**（本地缓存扛住绝大部分读）、key 加随机后缀打散到多分片、读写分离、限流。
- 发现手段：redis-cli --bigkeys / --hotkeys、MONITOR（谨慎）、慢日志、自研采样上报。
- 原则：预防为主——key 设计时就控制粒度、避免全量聚合、热点业务提前规划多级缓存。

评分要点

- **优秀**：讲清危害机理（单线程阻塞/分片倾斜）、local cache 扛热 key、unlink 异步删除等具体手段。
- **及格**：能说清两者概念与基本治理方向。
- **不足**：只知道"大 key 不好"。

常见追问：1) 本地缓存与 Redis 的一致性怎么处理？2) 集群模式下热 key 为什么更危险？

【A2-23】【L2】Nacos 注册与配置中心

题目：Nacos 怎么做服务注册发现和配置管理？临时实例和持久实例有什么区别？

思考方向：服务注册/心跳/健康检查 + 配置动态刷新，CP/AP 模式差异。

考察点：微服务治理。

参考答案

- 注册发现：服务启动向 Nacos 注册实例信息，消费方拉取（+ 订阅推送）实例列表，通过 Ribbon/LoadBalancer 做客户端负载均衡调用。
- 健康检查：**临时实例**用客户端心跳（默认 5s），心跳丢失则剔除，Nacos 1.x 默认 AP（可用性优先，基于 Distro 协议 + 自研一致性）；**持久实例**由 Nacos 服务端主动探测（TCP/HTTP），不健康只标记不剔除，走 Raft 保证 CP。
- 配置中心：配置集中管理、按 namespace/group/dataId 隔离；@RefreshScope + Nacos 长轮询实现配置变更动态刷新，不用重启。
- 实践：环境用 namespace 隔离，配置按共享（common）+ 应用维度拆分，敏感信息安全加密。

评分要点

- **优秀**：讲清心跳 vs 主动探测、AP/CP 选择依据、长轮询动态刷新机制。
- **及格**：能说出注册发现流程与配置动态刷新。
- **不足**：只把 Nacos 当"配置文件的存储"。

常见追问：1) Nacos 集群脑裂怎么处理？2) 配置刷新失败怎么排查？

【A2-24】【L2】OpenFeign 原理

题目：OpenFeign 是怎么把接口调用变成 HTTP 请求的？超时和重试怎么配？

思考方向：动态代理 + 注解解析 + 负载均衡 + 编解码。

考察点：远程调用机制。

参考答案

- 原理：@FeignClient 接口在启动时被扫描，通过 JDK 动态代理生成实现；调用方法时解析 @RequestMapping 等注解拼出 HTTP 请求，经负载均衡器选择实例，再由 Encoder/Decoder 做序列化与响应解析。
- 集成：底层默认用 LoadBalancer 从注册中心取实例，可结合 Sentinel 做熔断、加拦截器传递鉴权 token/链路 ID。
- 配置：连接超时 connectTimeout、读超时 readTimeout（默认偏短需调整）、重试 Retryer（注意非幂等接口不能随便重试）；支持 GZIP 压缩、日志级别。
- 坑：Feign 与 Hystrix/Sentinel 的线程隔离会丢失 ThreadLocal（需用可传递的上下文或请求头），首次调用有懒加载耗时（可配饥饿加载）。

评分要点

- **优秀：**讲清动态代理到 HTTP 的转换链路，指出重试的幂等风险与上下文透传问题。
- **及格：**能说出动态代理 + 负载均衡 + 编解码的基本原理。
- **不足：**只知道“Feign 可以像调本地方法一样调远程”。

常见追问：1) Feign 和 RestTemplate 的区别？2) 非幂等接口的重试怎么防重复下单？

【A2-25】【L2】Gateway 路由、过滤器与限流

题目：Spring Cloud Gateway 的工作原理？怎么做统一鉴权和限流？

思考方向：基于 WebFlux 的响应式网关，路由断言 + 过滤器链。

考察点：网关设计与安全。

参考答案

- 原理：基于 Spring WebFlux + Reactor Netty 的响应式非阻塞网关；请求先匹配 Route（由断言 Predicate 决定，如 Path/Host/自定义），再经过 Pre → 转发 → Post 的过滤器链（GatewayFilter/GlobalFilter）。
- 统一鉴权：在 GlobalFilter 中校验 JWT/token → 解析用户身份 → 写入请求头（用户 ID/租户）透传给下游，未通过直接返回 401，避免每个服务重复校验。
- 限流：内置 RequestRateLimiter 基于 Redis + Lua 令牌桶（KeyResolver 按 IP/用户/IP+接口），超限返回 429；也可接 Sentinel 做更细的流控。
- 其他：灰度（按 header 路由）、重试、熔断、请求/响应日志、跨域统一处理。

评分要点

- **优秀：**讲清响应式过滤器链与执行顺序、JWT 透传设计、Redis 令牌桶限流的 Key 设计。
- **及格：**能说出路由断言 + 过滤器 + 限流的基本用法。
- **不足：**只知道“网关就是转发请求”。

常见追问：1) Gateway 为什么不用 Servlet 而用 WebFlux？2) 网关限流和 Sentinel 应用层限流的区别？

【A2-26】【L2】Sentinel 熔断降级与限流算法

题目：Sentinel 的滑动窗口是怎么实现的？熔断降级策略有哪些？限流、熔断、隔离有什么区别？

思考方向：滑动窗口统计 + 三种熔断策略 + 三种保护手段定位。

考察点：服务治理、稳定性设计。

参考答案

- 滑动窗口：用 LeapArray 把时间切成多个等长小窗口（如 1s 分 2 个 500ms 窗口），统计时滑动聚合最近 N 个窗口的 QPS/异常数，比固定窗口更平滑、避免临界突变。
- 熔断策略：① 慢调用比例（RT 超阈值占比）；② 异常比例；③ 异常数；达到阈值进入 OPEN 状态，经时间窗后半开（HALF_OPEN）探测，成功则恢复。
- 三者区别：**限流**控制入口流量（保护系统不被打垮）；**熔断**在下游持续失败时快速失败（防止雪崩、避免无谓等待）；**隔离**（线程池/信号量）把不同依赖的资源隔开，一个慢不影响其他。三者组合使用。
- 降级：熔断后走 fallback（返回默认值/缓存/友好提示/转人工），保证核心链路可用。

评分要点

- **优秀**：讲清滑动窗口聚合原理、三种熔断策略与半开恢复、三者定位区别。
- **及格**：能说出限流/熔断/降级的概念与常见策略。
- **不足**：把限流和熔断混为一谈。

常见追问：1) 令牌桶和漏桶的区别？2) 熔断阈值怎么定，定错了会怎样？

【A2-27】【L3】分布式事务方案

题目：跨服务下单扣库存，怎么保证一致性？2PC、TCC、Seata AT、可靠消息、Saga 怎么选？

思考方向：强一致 vs 最终一致，按业务容忍度选择。

考察点：分布式事务选型（高频）。

参考答案

- **2PC/XA**：强一致但同步阻塞、性能差、协调者单点，互联网场景少用。
- **TCC**：Try-Confirm-Cancel 三段，业务侵入大，但性能与可控性好；需处理空回滚、悬挂、幂等三大问题。
- **Seata AT**：无侵入（基于 SQL 解析自动生成 undo log + 全局锁），两阶段提交，适合大多数业务；缺点是有全局锁、热点数据有争用。
- **可靠消息最终一致**：本地事务 + 消息表/事务消息，下游消费保证幂等，实现最终一致，性能好，适合跨服务异步场景（如支付成功后通知发货）。
- **Saga**：长事务拆成多步本地事务 + 补偿，适合流程长、可补偿的业务。选型：优先"最终一致 + 幂等 + 补偿"，强一致需求极强才上 TCC/Seata。

评分要点

- **优秀**：给出选型决策树，讲清 TCC 三大坑（空回滚/悬挂/幂等）与可靠消息的落地方式。
- **及格**：能区分强一致/最终一致并说出 2~3 种方案适用场景。
- **不足**：只会说"用 Seata"。

常见追问：1) 可靠消息方案怎么保证消息不丢不重？2) 为什么不直接都用 TCC？

【A2-28】【L2】RabbitMQ 可靠投递、幂等与延迟队列

题目：你怎么保证消息不丢、不重复消费？延迟任务怎么实现？

思考方向：三段（生产者→Broker→消费者）分别保证可靠性。

考察点：消息中间件实战。

参考答案

- **不丢**：① 生产者开启 confirm 确认 + 失败重试（本地消息表兜底）；② Broker 持久化（交换机/队列/消息都设持久化，镜像/quorum 队列防节点故障）；③ 消费者手动 ack，处理成功才确认，失败 requeue 或进死信。
- **不重复**：MQ 只能保证 at-least-once，重复几乎不可避免 → **消费端幂等**：用业务唯一 ID + Redis/数据库唯一索引/状态机做去重，保证同一消息多次消费结果一致。
- **延迟任务**：① TTL + 死信队列（消息过期进死信，消费者监听死信）；② 延迟插件 rabbitmq-delayed-message-exchange；③ 更复杂定时调度用 xxl-job。项目里用于订单超时取消、批量发

货的延迟重试。

- 积压/失败：配置死信队列、消费限流（prefetch）、失败重试次数 + 人工补偿入口。

评分要点

- **优秀**：讲清三段可靠性的具体手段 + 幂等的实现方式 + 延迟队列两种实现与选择依据。
- **及格**：能说出 confirm/ack/持久化与消费幂等。
- **不足**：认为"MQ 天然不丢不重"。

常见追问：1) 手动 ack 时业务处理完宕机了怎么办？2) 消息顺序性怎么保证？

【A2-29】【L3】消息积压排查与高可用原理

题目：线上消息积压了几百万条，怎么应急处理？Kafka 为什么吞吐高？

思考方向：先止血（扩容消费者/临时转存）再根治（找消费慢的根因）。

考察点：故障应急、中间件原理。

参考答案

- 积压应急：① 先定位是消费者数不足（队列数 < 消费者数）还是单条处理太慢（慢查询/外部调用超时）；② 紧急扩容消费者（增加并发消费线程/实例、增加队列与分区数）；③ 极端情况写临时“搬运消费者”把消息快速转存到新队列再慢慢消费；④ 临时降级非核心处理逻辑（跳过日志/统计），先追平积压。
- 根治：优化消费逻辑（批量、异步、加缓存）、设置消费限流避免拖垮下游、加监控告警（堆积量/消费 TPS）。
- Kafka 高吞吐原理：顺序写磁盘 + 页缓存（Page Cache）；**零拷贝**（sendfile 跳过用户态）；批量发送 + 压缩；分区并行。可靠性靠 ack 机制（0/1/all）与 ISR 副本同步。
- 选型：Kafka（高吞吐日志/流处理）、RocketMQ（事务消息、顺序消息、电商友好）、RabbitMQ（路由灵活、延迟队列、中小规模）。

评分要点

- **优秀**：给出“止血 + 根治”两步应急方案，讲清零拷贝/顺序写/ISR 原理与三者选型。
- **及格**：能说出扩容消费者 + 优化消费逻辑，知道 Kafka 顺序写与零拷贝。
- **不足**：只会说“重启消费者”。

常见追问：1) 消费者扩到比分区数还多有用吗？2) ack=all 的代价是什么？

方向三：Python AI 应用开发（A3，共 6 题）

【A3-01】【L2】Python 异步编程与 GIL

题目：Python 的 GIL 是什么？asyncio 和多线程在 AI 应用里怎么选？

思考方向：GIL 限制的是“多线程并行执行 CPU 密集代码”，不限制 IO 并发。

考察点：Python 并发模型。

参考答案

- GIL（全局解释器锁）：CPython 同一时刻只允许一个线程执行字节码，所以多线程无法并行跑满多核（CPU 密集无收益）。
- 但 IO 等待时会释放 GIL，所以**多线程/协程对 IO 密集型仍有效**；CPU 密集需多进程（multiprocessing）。
- asyncio：单线程事件循环 + 协程，靠 await 让出控制权，开销远小于线程，适合“大量并发等待外部 IO”的场景（AI 应用里最典型：并发调用大模型 API、向量库查询、工具调用）。
- 选择：AI 服务多为 IO 密集 → asyncio + 异步 HTTP 客户端（httpx/aiohttp）最合适；与同步 SDK 混用时要避免在协程里调用阻塞代码（用 run_in_executor 包装）。

评分要点

- **优秀**：讲清 GIL 只在字节码执行期生效、IO 会释放、以及"协程里别调阻塞函数"的坑。
- **及格**：能说出 GIL 概念与 asyncio 适合 IO 密集。
- **不足**：认为"Python 多线程没用/完全没用"。

常见追问：1) 协程里调用了阻塞库会怎样？2) 多进程方案在 AI 服务里的成本？

【A3-02】【L2】LangChain 核心抽象

题目：LangChain 的核心组件有哪些？LCEL 是什么，为什么用管道符写法？

思考方向：把"提示词→模型→解析器→工具→记忆"抽象成可组合的组件。

考察点：LangChain 生态认知。

参考答案

- 核心抽象：Models (LLM/ChatModel/Embeddings)、Prompts (模板 + 变量)、Output Parsers (结构化输出)、Tools & Agents、Memory (会话记忆)、Retrievers / VectorStores (RAG)、Callbacks (观测)。
- LCEL (LangChain Expression Language)：用 `prompt | llm | parser` 的管道符号把组件串成 Runnable 链；每个节点实现同一套 Runnable 接口 (invoke/stream/batch/ainvoke)，因此天然支持流式、批量、异步与重试/回退 (with_fallbacks)。
- 价值：声明式组合、可复用、便于调试 (回调/追踪)，替代了过去晦涩的 Chain 子类继承体系。

评分要点

- **优秀**：讲清 Runnable 统一接口带来的"流式/异步/回退"能力，能对比旧版 Chain。
- **及格**：能列出主要组件并说清 LCEL 管道式组合。
- **不足**：只会说"LangChain 是用来做 RAG 的库"。

常见追问：1) LCEL 怎么加流式输出？2) LangChain 的 Memory 和 LangGraph 的状态有何区别？

【A3-03】【L2】LangGraph 状态、节点、边与条件路由

题目：LangGraph 的 StateGraph 是什么？节点、边、条件边怎么用？和 LangChain 的 Chain 有什么本质区别？

思考方向：Chain 是"直线管道"，LangGraph 是"可循环的状态机图"。

考察点：Agent workflow 编排。

参考答案

- StateGraph：以共享的 State (TypedDict/自定义，可定义 reducer 合并策略) 为核心的**有向图**；节点是函数 (输入 State、返回 State 更新)，普通边是固定流转，**条件边**按 State 判断走哪条分支。
- 与 Chain 的区别：Chain 是单向无环的管道，无法表达循环、分支与人工介入；LangGraph 支持**循环** (如"检索→评估→不达标则改写再检索")、分支路由、以及运行中断 (interrupt)，这才是 Agent 所需的控制流。
- 实践：多步 Agent 工作流 (意图识别→工具选择→执行→结果校验→不足则重规划) 非常适合用图表达；节点职责单一、便于测试与观测。

评分要点

- **优秀**：讲清 State 的 reducer 机制、循环与条件边的必要性，能举"检索-评估-重试"的例子。
- **及格**：能说清节点/边/条件边与"图可循环、Chain 不可"的区别。
- **不足**：以为 LangGraph 只是"LangChain 的新版本"。

常见追问：1) State 并发更新怎么合并？2) 怎么防止图里出现死循环？

【A3-04】【L3】LangGraph Checkpointer 与人工介入

题目：LangGraph 怎么做状态持久化与断点续跑？interrupt 人工介入如何实现？

思考方向：Checkpoint 在每个节点后存档，thread_id 标识一次会话。

考察点：有状态编排、人机协同。

参考答案

- Checkpointer：配置（MemorySaver / SqliteSaver / PostgresSaver 等）后，图在**每个节点执行后自动保存 State 快照**，以 thread_id 作为会话标识；同一个 thread_id 可恢复、可回溯历史（time travel）。
- 价值：长流程中断后从断点继续、支持重放调试、多轮会话状态不丢；生产上 Checkpointer 要持久化到数据库/Redis，避免内存态丢失。
- 人工介入：在节点内调用 `interrupt(payload)` 暂停图并抛出待人工处理的信息，外部审批后用 `Command(resume=...)` 恢复执行；适用于"高风险工具调用审批""人工确认后再提交"等场景。
- 与 RAG 人工接管的关系：前者是"流程内的人审"，后者是"低置信兜底转人工"，可组合使用。

评分要点

- **优秀：**讲清"节点级快照 + thread_id 恢复 + interrupt/resume"的完整机制与生产持久化考量。
- **及格：**能说出 Checkpointer 保存状态、interrupt 暂停等人工。
- **不足：**不知道 LangGraph 有状态持久化能力。

常见追问：1) 恢复执行时外部依赖已变化怎么办？2) Checkpoint 数据量大了怎么清理？

【A3-05】【L2】Celery 异步任务与接口限流降级

题目：AI 服务里哪些逻辑应该异步化？Celery 怎么用？接口限流降级怎么做？

思考方向：把"耗时长且不阻塞用户"的任务拆出去，用消息队列解耦。

考察点：服务端工程化。

参考答案

- 应异步化的场景：文档解析/向量化入库、批量评测、报告导出、长时 Agent 任务、外部大模型批量调用；给用户先返回 task_id，前端轮询/推送进度。
- Celery：Producer 投递任务到 Broker（Redis/RabbitMQ），Worker 消费执行，Result Backend 存结果；支持重试、超时、定时（beat）、路由到不同队列。注意任务幂等与失败重试上限。
- 限流降级：按用户/IP/接口维度做令牌桶限流（Redis + Lua），超限返回 429；大模型调用设置超时与重试上限，失败降级到小模型/缓存/兜底话术；保护下游和成本。
- 与项目对应：文档入库、评测、批量导出这些"重活"都不应放在请求线程里同步做。

评分要点

- **优秀：**能准确判断哪些该异步化，讲清 Celery 队列/重试/幂等与限流降级的组合。
- **及格：**能说出用 Celery 处理耗时任务 + Redis 限流。
- **不足：**把所有逻辑都塞在接口里同步执行。

常见追问：1) 异步任务失败怎么通知用户？2) Celery 任务重复执行怎么防？

【A3-06】【L2】微调：SFT / LoRA / QLoRA（非必背，看个人情况选择）

题目：讲一下 SFT、LoRA、QLoRA 的关系；LoRA 的 rank 和 alpha 是什么？微调后为什么会"灾难性遗忘"？

思考方向：从"全参微调太贵"引出参数高效微调（PEFT）。

考察点：微调基础（RAG 岗也常问，属于补强项）。

参考答案

- SFT：用标注的"指令-回答"数据做监督微调，让基座模型学会遵循指令与领域风格；全参微调显存/算力成本高。

- LoRA: 冻结原权重, 在注意力/FFN 旁路插入低秩矩阵 $A \times B$ 近似更新量 ΔW , 只训练这部分小参数; **rank r** 决定低秩矩阵的容量 (越大越能拟合、参数越多), **alpha** 是缩放系数 (常按 α/r 缩放, 控制更新幅度)。训练后可合并回原权重, 推理无额外延迟。
- QLoRA: 把基座模型权重用 4bit 量化 (NF4) 再加 LoRA 训练, 显存大幅下降, 可单卡微调大模型; 代价是训练稍慢、量化有精度损失。
- 灾难性遗忘: 只在小领域数据上微调会破坏模型原有的通用能力; 缓解手段是混入通用数据、降低学习率/训练轮数、用 LoRA 只更新少量参数。
- 选型补充: 知识类用 RAG, 行为/风格/格式类优先 Prompt, 确需固化能力或降本 (训小模型) 才微调。

评分要点

- **优秀**: 讲清 LoRA 的低秩旁路原理、 r/α 作用、QLoRA 的 4bit 量化, 以及遗忘的缓解; 能给出“何时该微调”的判断。
- **及格**: 知道 SFT/LoRA/QLoRA 是什么、LoRA 只训少量参数、QLoRA 省显存。
- **不足**: 完全不了解微调 (简历未体现, 但面试常问, 需补齐)。

常见追问: 1) LoRA 加载到哪些层效果最好? 2) 微调效果不达标怎么排查 (数据/超参/评测)?

方向四: 前端工程化与全栈 (A4, 共 8 题)

【A4-01】【L2】Vue3 响应式原理

题目: Vue3 的响应式是怎么实现的? 和 Vue2 的 `Object.defineProperty` 比有什么优势?

思考方向: Proxy 拦截读写下依赖收集与派发更新, 对比 `defineProperty` 的局限。

考察点: 框架原理 (简历写“了解 Vue 源码”必问)。

参考答案

- Vue3 用 **Proxy** 代理整个对象, 配合 `effect()` 做依赖收集 (track) 与触发更新 (trigger), 依赖关系存在 `WeakMap` $\rightarrow$ `Map` $\rightarrow$ `Set` 的 `targetMap` 中。
- 对比 Vue2 的 `Object.defineProperty`: ① 只能劫持已存在的属性, 新增/删除属性需 `$set`, Proxy 能拦截 `get/set/deleteProperty` 等 13 种操作; ② 无法直接监听数组下标与 `length` 变化 (Vue2 靠重写数组方法), Proxy 天然支持; ③ 惰性递归代理 (访问到才代理嵌套对象), 初始化性能更好。
- 衍生: `ref` 用 `RefImpl` 类 + `get value/set value` 拦截, `reactive` 用 Proxy; `computed` 基于 `effect` 惰性求值 + 缓存。

评分要点

- **优秀**: 讲出 `targetMap` 依赖结构、Proxy 相较 `defineProperty` 的具体优势、`ref/reactive/computed` 的实现差异。
- **及格**: 能说出 Vue3 用 Proxy、Vue2 用 `defineProperty` 及主要差异。
- **不足**: 只会说“Vue3 性能更好”。

常见追问: 1) 为什么 Vue3 还需要 `ref`? 2) 依赖收集在什么时机进行?

【A4-02】【L1】虚拟 DOM 与 diff

题目: 虚拟 DOM 是什么? 为什么需要它? diff 算法的大致流程?

思考方向: 用 JS 对象描述 DOM, 减少真实 DOM 操作, 跨平台。

考察点: 前端框架基础。

参考答案

- 虚拟 DOM 是用 JS 对象 (VNode) 描述真实 DOM 结构, 更新时先生成新 VNode 树再与旧树 diff, 只把差异应用到真实 DOM, 减少昂贵 DOM 操作次数, 同时让渲染逻辑与平台解耦 (可渲染到小程序/SSR)。

- diff 优化：同层比较（不跨层级）、Vue3 用 patchFlag 标记动态节点跳过静态内容、最长递增子序列复用 key 节点减少移动、Vue2 双端比较。
- 关键点：**key 的作用**是标识节点身份，用 index 作 key 在列表增删时会导致状态错位与多余更新。

评分要点

- **优秀**：讲清"同层比较 + patchFlag 静态提升 + 最长递增子序列"等优化与 key 的正确用法。
- **及格**：能说清虚拟 DOM 的作用与 diff 同层比较。
- **不足**：认为"虚拟 DOM 一定比直接操作 DOM 快"。

常见追问：1) 为什么用 index 当 key 不好？2) 虚拟 DOM 的缺点是什么？

【A4-03】【L2】首屏性能优化体系

题目：你项目把首屏加载降低 30%-50%，具体做了哪些优化？怎么衡量效果？

思考方向：按"资源体积、加载策略、渲染性能、接口"四层展开。

考察点：性能优化实战（简历量化点，必被深挖）。

参考答案

- 资源体积：路由懒加载 + 代码分割、Tree Shaking、按需引入（如 ECharts 按需）、gzip/brotli 压缩、图片压缩与 CDN、拆分 vendor。
- 加载策略：资源预加载/预连接、首屏关键资源优先、非关键组件异步、HTTP 缓存（强缓存 + 协商缓存）。
- 渲染性能：长列表虚拟滚动、图片懒加载（IntersectionObserver）、接口并发控制与请求缓存、骨架屏/loading 提升体感。
- 接口：合并请求、SSR/预渲染（若需 SEO）、接口响应裁剪。
- 衡量：Lighthouse/Performance 面板看 FCP/LCP/TTI、Performance API 打点统计真实用户指标（结合项目 Sentry 埋点）；对比优化前后数据。

评分要点

- **优秀**：分层次讲清手段，并说明"用什么指标衡量、怎么拿到 30%-50% 这个数字"。
- **及格**：能说出懒加载、压缩、虚拟滚动、缓存等常见手段。
- **不足**：只会说"图片压缩一下"。

常见追问：1) 首屏指标 FCP 和 LCP 的区别？2) 优化后数据怎么归因到具体手段？

【A4-04】【L2】瀑布流组件实现

题目：你的瀑布流组件怎么实现的？PC/移动端动态切换、图片预加载与懒加载切换怎么处理？

思考方向：计算最短列高度插入，配合容器宽度与列数动态计算。

考察点：组件抽象、布局算法。

参考答案

- 核心算法：等分列（根据容器宽度与列数算每列宽度）→ 维护各列当前高度数组 → 每个新卡片插入当前最矮列 → 更新该列高度；用绝对定位或 flex 分列两种实现（分列更简单、绝对定位更省 DOM）。
- 响应式：监听容器 resize（ResizeObserver），重新计算列数（PC 多列、移动端两列）与布局；节流处理避免频繁重排。
- 图片：用占位宽高比先撑开高度避免抖动；IntersectionObserver 实现懒加载，进入视口前可预加载下一屏；加载完成回调触发重新布局（图片未加载完高度不准的经典问题）。
- 工程化：做成组件暴露 props（列数/间距/断点）与 slot，配 README 与示例页，作为组件库一员发布到私有 npm。

评分要点

- **优秀**：讲清"最短列插入"算法、图片未加载导致高度抖动这一真实难点及解法。
- **及格**：能说清分列 + 插入最矮列的基本思路。
- **不足**：只能说"用 CSS column 实现"。

常见追问：1) 大量卡片时性能怎么保证？2) 数据滚动加载（无限滚动）怎么和瀑布流配合？

【A4-05】【L1】IntersectionObserver 与懒加载

题目：你怎么实现图片/长列表懒加载？IntersectionObserver 相比 scroll 监听事件有什么优势？

思考方向：浏览器原生交叉观察，异步回调、性能更好。

考察点：浏览器 API、性能意识。

参考答案

- 实现：创建 IntersectionObserver，注册需要观察的元素；当元素进入视口（intersectionRatio > 0）时把 data-src 赋给 src 并 unobserve；可设 rootMargin 提前预加载。
- 优势：不依赖 scroll 事件，由浏览器在渲染线程外异步计算，**不会因高频 scroll 触发大量回调导致卡顿**；无需手动计算 offsetTop/getBoundingClientRect（后者会触发重排）。
- 工程：封装成 Vue 自定义指令 v-lazy，业务侧直接用；配合 loading 占位图与失败兜底；兼容性差的环境降级为 scroll + 节流。
- 价值：项目里大量图片（漫画/商品）场景显著减少首屏请求与流量。

评分要点

- **优秀**：讲清"异步回调不阻塞主线程"的性能优势，并说明封装成指令与降级方案。
- **及格**：能说清 IntersectionObserver 的用法与基本优势。
- **不足**：只会用 scroll 事件 + 节流。

常见追问：1) rootMargin 有什么用？2) 指令和组件的选择依据？

【A4-06】【L2】Render 函数动态 VNode 与命令式组件

题目：你的 message/confirm 组件用 Render 函数实现，具体怎么做的？为什么这类组件适合命令式调用？

思考方向：把组件实例挂载到动态创建的容器上，返回 Promise。

考察点：Vue 渲染机制、API 设计能力。

参考答案

- 命令式调用：`Message.success('xx')` 直接在 JS 里弹出提示，不必在模板里写标签；适合 toast/确认框这类"用完即走、与页面结构无关"的 UI。
- 实现：在调用时动态 createElement 一个容器挂到 body → 用 render(h(Comp, props), container) 或 createVNode + render 把组件实例渲染进去 → 组件内部通过 Transition 做动画 → 关闭时调用 render(null, container) 卸载并移除容器节点，避免 DOM 泄漏。
- confirm 返回 Promise，resolve/reject 对应确定/取消，方便 await 写法；多个实例用容器数组管理、指定 zIndex 层级；同一时刻可只保留一个实例（单例）或允许多个（栈管理）。
- 关键：手动卸载 + 清理引用是这类组件最容易漏的内存泄漏点。

评分要点

- **优秀**：讲清动态挂载/卸载流程、Promise 化 API 设计、以及 DOM 泄漏的防范。
- **及格**：能说清"动态创建容器 + render 渲染 + 用完卸载"。
- **不足**：只会说"用 Render 函数创建 VNode"。

常见追问：1) 用 h 函数和用 template 有性能差异吗？2) 多个 message 同时弹出怎么管理？

【A4-07】【L2】Webpack 构建优化

题目：你在项目里做的 Webpack 优化有哪些？Tree Shaking 生效的前置条件是什么？

思考方向：体积、速度两条线；Tree Shaking 依赖 ESM 静态分析。

考察点：构建工具实战。

参考答案

- 体积优化：splitChunks 拆分 vendor 与业务代码、Tree Shaking、gzip/brotli 压缩、按需加载（路由级）、Externals/CDN、去除无用依赖与包分析（webpack-bundle-analyzer）。
- 速度优化：持久化缓存/文件系统缓存、多线程（thread-loader）、缩小 resolve 范围与 exclude、DLL/预先编译依赖、开发环境减少 sourcemap 开销。
- Tree Shaking 前置条件：① 必须是 **ES Module** 语法（import/export），CommonJS 无法静态分析；② 生产模式 + `sideEffects:false` 标记；③ 代码无副作用、未被打包工具“保住”；④ 库需要提供 ESM 产物（看 package.json 的 module 字段）。
- ECharts 按需：import 具体图表模块而非整体引入，减小打包体积。

评分要点

- **优秀：**讲清 Tree Shaking 的多个前置条件、splitChunks 策略与缓存提速，能结合体积/速度两条线。
- **及格：**能说出 splitChunks、gzip、Tree Shaking 等常见手段。
- **不足：**只会说“开了 gzip”。

常见追问：1) Vite 为什么开发快？2) 构建体积和运行性能有冲突吗？

【A4-08】【L2】小程序性能与多接口并发

题目：微信小程序的包体积和性能怎么优化？多个接口并发请求怎么管理？

思考方向：主包/分包、组件化、setData 优化、请求封装。

考察点：小程序工程化。

参考答案

- 包体积：主包只放核心页面，非核心页面分包/独立分包、分包预下载；图片走 CDN 不打包；代码压缩与资源压缩；及时清理无用代码与重复依赖。
- 渲染性能：减少 setData 的数据量与频次（只传变化的最小字段，避免大对象整体 setData）、长列表用官方 recycle-view 或虚拟列表、避免频繁跨线程通信。
- 组件化：封装商品卡片、轮播图、分类导航等通用组件（项目里已做），复用提升可维护性。
- 多接口并发：用 `Promise.all` 并发拉取并统一 loading/错误兜底，避免“接口瀑布”串行等待；对失败接口做局部降级（不阻塞整体渲染）；封装统一请求层处理鉴权、重试、loading 与错误提示。

评分要点

- **优秀：**讲清分包策略、setData 的性能陷阱、以及“并发 + 局部降级”的接口管理。
- **及格：**能说出分包、setData 优化、Promise.all 并发。
- **不足：**只会说“用小程序框架开发”。

常见追问：1) 小程序主包 2M 限制怎么突破？2) 接口全部失败时页面该怎么表现？

B. 项目深挖题 共 17 题

项目一：企业智能问答 Agent（B1，5 题）

【B1-01】【L3】为什么用“混合召回 + RRF + Rerank”三件套

题目：你简历写"BM25 + 向量混合召回 → RRF 融合 → Rerank 精排"，为什么不是只用向量检索？这三步各自解决什么问题、怎么证明有效？

思考方向：从销售的提问特点（既有口语化描述、又有商品型号/健康术语）切入，说明单一召回的漏召问题。

考察点：RAG 检索策略设计、方案论证能力。

参考答案

- 销售提问里既有口语化表达（"这个吃了会不会上火"→需语义理解），又有精确实体（商品型号、成分名称、代码类名词→需关键词精确匹配）。纯向量对实体/型号不敏感，纯 BM25 不解语义，都会漏召。
- 三步分工：混合召回"提召回"（宁可多召回不能漏）→ RRF"把两路按排名融合"（规避量纲问题，鲁棒）→ Rerank"精排提相关性"（cross-encoder 交互打分，把最相关的顶上来）。
- 有效性证明：用离线评测集对比"纯向量 / 纯BM25 / 混合 / 混合+Rerank"几组配置的 Recall@K、MRR、Faithfulness，看指标提升；再用线上"一次答准率"验证。
- 代价意识：Rerank 会引入额外延迟，本项目 2s 内可接受，若延迟敏感需缩减候选数或换轻量模型。

评分要点

- **优秀：**能用业务问题说明单一召回的缺陷，讲清三步分工与"用指标对比验证"的方法，有延迟代价意识。
- **及格：**讲清混合召回互补、RRF 融合、Rerank 精排的作用。
- **不足：**只说"这样效果更好"，讲不出验证方式与代价。

常见追问：1) 如果向量召回结果全错，RRF 能救回来吗？2) Rerank 候选数（TopK）定多少，依据是什么？

【B1-02】【L3】语义分块 + 父子文档的参数与"85% 一次答准率"的口径

题目：你的"语义分块 + 父子文档"具体参数怎么定的？简历里"高频问题一次答准率提升至约 85%"，这个数是怎么统计出来的？

思考方向：诚实说明统计口径（样本、判定方式、分母），并解释分块实验如何做。

考察点：数据可信度、实验方法（高频追问"数据来源"）。

参考答案

- 口径必须说清：从业务真实问题里抽 N 条（如 200~500 条）高频问题，人工/双人标注标准答案，**答对=答案与标准答案要点一致且无编造**，"一次答准率=首轮回答即正确的问题数 / 总问题数"；分母是测试集样本而非线上全部对话。
- 分块实验：控制变量，只改策略（固定分块 vs 语义分块、chunk size 300/500/800、overlap 10%/20%），跑同一评测集看 Recall@K / MRR / Faithfulness，选 Pareto 最优；父子文档用"子块（约 200~400 token）检索 + 父块（段落/章节）喂模型"。
- 对比基线：优化前（固定分块 + 纯向量）的答准率作为基线，优化后提升到 85%，能说清每一步贡献多少。
- 坦诚：若 85% 是"高频问题"这一子集的口径而非全量，务必主动说明，避免面试官追问时被认为夸大。

评分要点

- **优秀：**能准确说出统计口径、样本量与优化前后对比，主动交代口径边界。
- **及格：**能说清"用评测集抽样 + 人工判定"的统计方式。
- **不足：**支支吾吾说"大概估的""感觉这样差不多"，是扣分重灾区。

常见追问：1) 标注由谁做、一致性怎么保证？2) 评测集会不会被反复调参"过拟合"？

【B1-03】【L2】"平均检索响应延迟控制在 2s 以内"怎么做到的

题目：全链路（改写 + 双路召回 + RRF + Rerank + 大模型生成）要在 2s 内完成，你怎么优化的？瓶颈在哪？

思考方向：拆解各阶段耗时，定位大头（通常是模型生成与 Rerank）。

考察点：性能优化、链路分析能力。

参考答案

- 先量化各段耗时（埋点打点）：query 改写、向量检索、BM25、RRF、Rerank、模型生成，找出大头。
- 优化手段：① 双路召回**并行**执行而非串行；② 缓存（热点问题语义缓存直接返回、Embedding 结果缓存）；③ 用流式输出让"首 token 时间"替代"全量生成时间"，用户感知延迟大幅下降；④ 控制 Rerank 候选数与模型大小；⑤ 向量库建 HNSW 索引、Metadata 前置过滤减少扫描；⑥ 模型侧用更快的模型或小模型路由。
- 说明：2s 是"检索链路 + 首字返回"的口径还是"完全生成完"的口径，要讲清楚——这是面试官最可能追问的点。

评分要点

- **优秀：**先打点定位瓶颈再谈优化，能讲清流式与缓存的作用，并主动澄清延迟口径。
- **及格：**能说出并行、缓存、流式、索引优化等手段。
- **不足：**只答"用了缓存"，没有量化。

常见追问：1) Rerank 占了多少毫秒？2) 缓存命中率和一致性怎么处理？

【B1-04】【L2】Function Calling 打通线上代码仓库 + MCP 接入层

题目：知识库命中不足时自动调用"线上代码仓库/业务接口"，这个工具怎么设计的？MCP 封装又带来了什么？

思考方向：工具描述设计、权限与安全、何时触发。

考察点：工具调用落地、工程化。

参考答案

- 工具设计：明确 name/description/参数 schema（如代码仓库检索需要 repo、关键词、分支等参数），描述里写清"什么时候该用这个工具"，直接决定模型选工具的准确率；返回值做截断与结构化，避免超长上下文。
- 触发时机：RAG 检索低于阈值或问题涉及"代码/实时数据"时，由 Agent 决策调用；工具结果与大模型生成结果共同组织答案。
- 安全：代码仓库**只读**、限定可访问仓库范围、鉴权走服务端（不把凭证交给模型）、限流防刷；返回内容做敏感信息过滤。
- MCP 的价值：把模型、工具、数据源统一到标准化接入层，新增工具只需起一个 MCP Server，宿主端无需改代码；对外暴露标准化问答接口，便于与其他业务系统集成。

评分要点

- **优秀：**讲清工具描述对选择准确率的影响、只读+服务端鉴权的安全设计、MCP 标准化接入带来的可扩展性。
- **及格：**能说清工具定义方式与"命中不足才调用"的逻辑。
- **不足：**只说"配了个工具让它去查代码"。

常见追问：1) 工具返回结果太长怎么处理？2) 模型选错工具怎么办，有没有兜底？

【B1-05】【L3】ReAct + Plan-and-Execute + 持久化记忆如何协同

题目：你项目同时提到 ReAct、Plan-and-Execute 和持久化记忆，三者在一个问答 Agent 里怎么配合？会不会过度设计？

思考方向：分层使用：简单问题走轻路径、复杂任务走规划路径、记忆做横向支撑。

考察点：架构合理性判断、避免炫技。

参考答案

- 流程：意图识别先做分流——简单知识问答直接走"检索→生成"轻路径（快）；多步任务（如"查出某商品近三月销量并对比趋势"）才升级为 Plan-and-Execute，规划出多个子步骤（查询/统计/汇总），每步内部用 ReAct 调工具。
- 记忆贯穿：短期记忆（会话上下文，Redis 按 sessionId 保证多轮指代消解；长期记忆（用户偏好、历史重要事实）存向量库，按需召回注入，实现个性化。
- 避免过度设计：默认走最简单的可行路径，只有任务确实需要多步时才动用规划；规划会带来额外模型调用（成本/延迟），必须用数据证明收益。
- 演进：这套分层为后续"流程自动化"预留了工具接入位，可平滑扩展。

评分要点

- **优秀**：讲清"分层路由 + 轻路径优先"的取舍，主动指出多步规划的额外成本。
- **及格**：能说清三者各自职责与基本配合方式。
- **不足**：把三个概念堆在一起，说不清谁在什么时机用。

常见追问：1) 意图识别用规则还是模型？误判怎么挽回？2) 长期记忆写入标准是什么？

项目二：企业 ERP 综合管理系统（B2，4 题）

【B2-01】【L2】订单查询 1s → 300ms：线程池并行 + Future 汇总

题目：你把串行调用客户/商品/发货信息改成并行，接口耗时从 1s 降到 300ms，具体怎么做的？线程池怎么配？为什么用 Future 而不是 CompletableFuture？

思考方向：先说明原来串行的耗时叠加，再讲并行改造与超时兜底。

考察点：并发优化实战、参数调优。

参考答案

- 原问题：三个远程调用串行，耗时 = $T_1 + T_2 + T_3 \approx 1s$ ；改为并行后耗时 $\approx \max(T_1, T_2, T_3) \approx 300ms$ ，本质是"把等待时间重叠起来"。
- 实现：用业务隔离的固定线程池提交三个 Callable，拿到 Future 后统一 get（带超时），再组装订单详情 DTO。
- 线程池配置：按"并发量 × 单任务耗时"估算核心数，IO 密集型可配较大（如 20~50），使用**有界队列 + 命名线程工厂 + 业务独立线程池**，避免与其他业务互相拖垮，也便于排查。
- Future vs CompletableFuture：Future 只能阻塞 get，无法链式编排；CompletableFuture 支持 thenCombine/allOf 等组合与异步回调，更适合复杂编排。**本项目三路无依赖，用 Future + 一起 get 已足够，不必过度设计**；若要加超时降级推荐 allOf + orTimeout。
- 兜底：单个子调用超时/失败时，该部分返回默认值而不是整体失败，保证主流程可用。

评分要点

- **优秀**：讲清并行原理与线程池配置依据、有界队列、业务隔离，并说明 Future/CompletableFuture 的选型理由。
- **及格**：能说清"并行替代串行、耗时取最大值"。
- **不足**：只说"用了多线程所以快了"，线程池参数说不清。

常见追问：1) 线程池打满会怎样，怎么监控？2) 三路里某一路超时，返回的数据怎么保持一致？

【B2-02】【L2】批量发货异步任务与失败重试

题目：批量订单发货拆成"任务批次 + 订单明细"、支持异步执行与失败重试，数据模型和状态机怎么设计的？重试怎么保证不重复发货？

思考方向：主任务表 + 明细表 + 状态字段 + 幂等键。

考察点：异步任务设计、幂等。

参考答案

- 数据模型：**批次表** (batch_id、总数、成功数、失败数、状态、创建人/时间) + **明细表** (batch_id、order_id、状态、失败原因、重试次数)。
- 流程：提交后立即返回 batch_id → 后台按批次消费（线程池/消息队列），逐条处理并更新明细状态与进度 → 前端按 batch_id 轮询进度。
- 幂等：每条明细有唯一键 (batch_id + order_id)，处理前先判状态（已成功则跳过），数据库唯一索引兜底，保证重试不会重复发货。
- 重试：只重试"失败态"明细，设置最大重试次数，超过转人工；失败原因结构化记录（库存不足/地址异常/系统超时），便于运营分类处理。
- 异常：批次级别做"可中断可续跑"，服务重启后能恢复未完成批次（状态扫描 + 定时补偿）。

评分要点

- **优秀**：讲清两级表结构与状态机、幂等键设计、重试与人工兜底、以及重启恢复方案。
- **及格**：能说清"批次+明细 + 异步执行 + 失败记录重试"。
- **不足**：只说"开了个线程池循环处理"，无状态与幂等设计。

常见追问：1) 批量任务很大时内存怎么控制？2) 同一个订单被两个批次包含怎么办？

【B2-03】【L3】多口径指标一致性（订单金额/签收业绩/财务利润）

题目：你负责统一处理"订单金额、签收业绩、财务利润"三个口径的指标，这三个口径为什么容易不一致？怎么保证数出一致？

思考方向：同一笔订单在不同口径下的计算规则不同（时间归属、退款、成本），关键是把规则收敛到一处。

考察点：业务理解、数据一致性治理。

参考答案

- 不一致根因：① 取值时间不同（下单时间 vs 签收时间 vs 财务确认时间）；② 是否含退款/优惠券/运费/赠品；③ 成本口径（含税/不含税、分摊方式）；④ 各模块各算一遍导致逻辑分叉。
- 治理：**口径统一到指标定义**——把三个指标的计算规则写成明确文档（公式 + 过滤条件 + 时间口径），在后端做统一的指标计算服务/视图，各模块调用同一计算逻辑，禁止各写各的 SQL。
- 数据层：明细表保留原始字段（金额、退款、成本、各关键时间点），指标在查询/聚合时按统一口径计算；对账场景用定时任务比对两套口径差异并告警。
- 验证：与财务/运营对账，抽样核对；差异可下钻到具体订单定位。

评分要点

- **优秀**：能从时间/退款/成本三方面说清口径差异根因，提出"统一指标定义 + 单一计算入口 + 对账告警"的治理方案。
- **及格**：能说清不同口径差异在哪，并知道要统一规则。
- **不足**：只答"按需求写的 SQL"，无治理思路。

常见追问：1) 口径变更后历史数据怎么办？2) 指标计算放数据库还是应用层，为什么？

【B2-04】【L2】Redis 主从读写分离在 ERP 的落地与坑

题目：ERP 里品牌/商品/订单高频缓存做读写分离，具体怎么改的？改造后遇到过什么问题？

思考方向：写主读从的收益 + 延迟/连接/序列化等工程问题。

考察点：缓存架构落地经验。

参考答案

- 落地：Spring 侧配置主从连接（Lettuce 主从读写分离或 Redisson 主从配置），读操作路由到从节点，写/删除走主节点；本地二级缓存（Caffeine）扛超热数据。
- 收益：主节点读压力下降、读吞吐提升，写不受读流量影响。
- 坑：① **主从延迟**导致刚写后立刻读可能读到旧值 → 对"刚操作完立刻回显"的场景强制读主（如订单详情）；② 缓存的 key 设计要包含租户/维度避免互相覆盖；③ 从节点故障需有降级（直连主节点）。

点)；④大对象序列化开销(可换JSON/压缩)。

- 配套：缓存失效策略(写后删缓存)、空值缓存防穿透、过期时间加随机防雪崩。

评分要点

- **优秀**：讲清读写路由配置与"读己之写"的处理，能举出真实踩坑。
- **及格**：能说清写主读从与延迟问题。
- **不足**：只会说"配一下就好了"。

常见追问：1) 读写分离后事务里的读怎么走？2) 怎么监控主从延迟？

项目三：美容美体网电商平台(B3, 4题)

【B3-01】【L3】"按商品ID加锁+二次检查"解决缓存击穿

题目：你把"全局锁"改成"按商品ID维度加锁+二次检查缓存"，为什么？全局锁有什么问题？二次检查为什么必要？

思考方向：锁粒度决定并发度；DCL思想的缓存版本。

考察点：并发控制粒度、缓存击穿治理。

参考答案

- 全局锁问题：所有商品共用一把锁，访问商品A的请求会阻塞访问商品B的请求 → **无关的请求互相排队，吞吐极低**；商家/用户量大时表现为整体接口变慢。
- 改造：锁key加商品ID维度(分布式锁key = lock:product:{id}，或本地锁分段)，不同商品互不阻塞，只保证"同一商品的回源串行"。
- 二次检查(DCL)：拿到锁后**再查一次缓存**——因为在等锁期间，前一个请求可能已经把数据加载进缓存了，此时直接返回即可，避免重复回源DB(否则N个请求只是串行地逐个查库，锁的意义打折)。
- 注意：锁要设过期时间防死锁；加锁失败可直接返回旧值/降级，不必全部阻塞；热点商品可用逻辑过期避免锁竞争。

评分要点

- **优秀**：讲清锁粒度与并发度的关系、二次检查的必要性，并提到逻辑过期等进阶方案。
- **及格**：能说清按ID加锁提升并发、二次检查防重复回源。
- **不足**：只会说"加了锁就好了"，说不清为什么要二次检查。

常见追问：1) 锁key用什么结构(Redis分布式锁vs本地锁)？2) 商品是爆款(超热)时这套方案还用吗？

【B3-02】【L2】下单接口限流降级：Redis原子计数+429

题目：你用"已认证用户ID+接口"为限流键、Redis原子计数实现多实例共享配额(每用户每10秒不超过2次)，具体怎么实现？为什么限流键要带接口？

思考方向：INCR+EXPIRE(或Lua原子)实现固定窗口计数，多实例共享。

考察点：限流实现、多实例一致性。

参考答案

- 实现：key = rate:{userId}:{api}，用Lua脚本原子执行"INCR+首次设置EXPIRE(10s)"，返回值>2则拒绝并返回HTTP 429+提示。多实例部署时共享同一个Redis计数，天然实现全局限额。
- 为什么带接口维度：不同接口的成本/风险不同(下单比查询敏感)，如果只按用户限流会"一刀切"，误伤正常查询；带接口维度可对高风险接口单独收紧。
- 注意点：①必须用Lua保证INCR与EXPIRE原子，否则进程崩溃会留下永不过期的key；②固定窗口存在临界问题(第10秒与第11秒各2次→2秒内4次)，严格场景可用滑动窗口/令牌

桶；③ 限流要区分"刷单"和"正常重试"，返回体要给出友好提示与重试引导；④ 已认证用户 ID 作为键前提是登录态可靠，未登录用户需降级用 IP + 设备指纹。

- 效果：拦截刷单与重复提交，保护下单链路与库存。

评分要点

- **优秀**：讲清 Lua 原子性、固定窗口临界问题、键维度设计理由与未登录降级。
- **及格**：能说清 Redis 计数 + 过期 + 多实例共享。
- **不足**：只会说"用 Redis 计数限流"，说不清原子性。

常见追问：1) 固定窗口临界突刺怎么解决？2) 限流误伤正常用户怎么申诉/白名单？

【B3-03】【L3】强制刷新分流 + 鉴权，避免"删缓存-重新回源"风暴

题目：你设计了"商品详情普通访问与强制刷新分流限流、刷新入口加鉴权"，要防的是什么风险？

思考方向：强制刷新是"删缓存 + 回源"的高危操作，被滥用会造成缓存雪崩/回源风暴。

考察点：缓存治理、安全与滥用防护。

参考答案

- 风险：强制刷新会**删掉缓存并触发回源**。如果普通用户可随意触发，恶意或误操作会瞬间把大量商品缓存打掉 → 请求全部打到 DB（回源风暴），即"人为制造的缓存雪崩"。
- 防护：① **接口分流**——普通访问走缓存不动缓存，强制刷新走独立入口；② **刷新入口加鉴权**（仅运营/管理员，RBAC 校验），普通用户不可调用；③ **限流**——对刷新入口按用户/接口限频，限制单位时间内的刷新次数；④ 刷新后温数据（异步重建）避免下次访问再回源。
- 延伸：类似思路适用于任何"人工触发缓存失效"的入口（后台运营工具都应有权限 + 限流 + 审计）。
- 价值：把"高风险运维动作"与"普通读流量"解耦，既保证运营能更新数据，又不让系统被误伤。

评分要点

- **优秀**：讲清"人为缓存雪崩"这一本质风险，给出分流+鉴权+限流+异步重建的组合防护。
- **及格**：能说清加权限和限流是为了防滥用。
- **不足**：只说"加了个刷新按钮"。

常见追问：1) 运营批量刷新上千商品怎么防？2) 刷新后新老缓存并发不一致怎么处理？

【B3-04】【L2】优惠券/满额赠品/藏宝岛分支的交易逻辑

题目：购物车结算里"满额赠品 + 优惠券可使用范围 + 藏宝岛特殊订单分支"，规则冲突时怎么处理？怎么防止优惠叠加出错？

思考方向：把促销规则做成可配置 + 明确优先级 + 服务端统一计算。

考察点：复杂业务规则设计。

参考答案

- 规则建模：把优惠券适用条件（订单金额门槛、商品范围/品类、时间、是否叠加）做成配置，结算时按**明确优先级顺序**依次计算（如 先商品级优惠 → 再订单级券 → 最后满额赠品），并给出可解释的"优惠明细"。
- 防错：**金额一律服务端计算**，前端只做展示与预估，防止篡改；下单时二次校验（券是否可用、库存是否足够、活动是否结束），校验通过才锁券/锁库存。
- 赠品：赠品数量与门槛校验在服务端做，赠品库存单独扣减，赠完即止；避免"先赠后算价"导致金额不对。
- 特殊分支（藏宝岛）：用策略模式按订单类型路由到不同处理链，避免把特殊逻辑散落在主流程里，降低回归风险。
- 一致性：下单 + 扣库存 + 核销券放在事务/最终一致方案里，超时未支付要回滚库存与券。

评分要点

- **优秀**：讲清规则优先级、服务端统一计算与二次校验、特殊分支策略模式隔离。
- **及格**：能说清优惠规则配置化 + 服务端校验 + 赠品门槛校验。
- **不足**：只会说"写了很多 if-else"。

常见追问：1) 优惠券可叠加时金额怎么算才不出错？2) 超卖/超赠怎么防？

项目四：自研组件库（B4，2 题）

【B4-01】【L2】组件库 0-1 建设：20+ 组件沉淀与私有 npm 发布

题目：组件库从 0 到 1 你做了哪些关键决策？多个后台系统与 H5 复用同一套组件，样式和构建怎么处理？

思考方向：项目结构、构建产物、样式隔离、版本与发布。

考察点：前端工程化与抽象能力。

参考答案

- **结构**：单仓库（monorepo 或单包 + 分类目录），按基础组件（button/dialog/message）与业务组件（瀑布流、虚拟任务栈）分层；提供统一入口与按需引入。
- **构建**：Vite/Rollup 打包 ES + CJS 双产物，外部依赖（Vue 等）作为 peerDependencies 不打包，产出类型声明（TS），支持按需加载与 Tree Shaking。
- **样式**：Tailwind 基础类 + 组件内部作用域样式；暴露 CSS 变量支持主题定制，避免污染业务全局样式。
- **发布与治理**：私有 npm 平台（如 Verdaccio）发布，语义化版本（SemVer），写 README + 示例页 + Changelog；用 ESLint/Prettier/Husky 保证提交规范与代码一致性。
- **价值**：20+ 组件让多个后台与 H5 项目复用，减少重复开发、统一交互体验。

评分要点

- **优秀**：讲清双产物/peerDependencies/Tree Shaking 等工程细节与版本治理。
- **及格**：能说清组件分层、私有 npm 发布、按需引入。
- **不足**：只会说"写了很多组件"。

常见追问：1) 组件升级如何不破坏使用方？2) 怎么做组件的自动化测试（覆盖率 85% 那类指标）？

【B4-02】【L2】虚拟任务栈 + KeepAlive 模拟 APP 跳转

题目：你用"自定义栈逻辑 + KeepAlive + Transform"实现 H5 页面跳转的 APP 效果，具体怎么做的？和浏览器原生 history 有什么冲突？

思考方向：维护自己的页面栈，配合转场动画与缓存。

考察点：SPA 路由与缓存机制。

参考答案

- **目标**：在 H5 里模拟原生 APP 的"进入向右滑入、返回向左滑出"以及"返回时保留上一页状态"。
- **实现**：维护一个页面栈数组（push/pop），每个页面记录方向（forward/back）；用 KeepAlive 缓存页面组件避免重建丢失滚动位置与表单；返回时先播动画再出栈，前进时反之；用 CSS transform + transition 做位移过渡。
- **与 history 冲突**：用户按浏览器返回键或物理返回会绕过自定义栈 → 需监听 popstate 同步栈状态（或用 hash/history 与自定义栈双向绑定），否则会出现"动画与路由不一致、页面栈错乱"。
- **缓存治理**：KeepAlive 无限缓存导致内存膨胀 → 设置最大缓存数（max）或只缓存必要页面，出栈时移除。

评分要点

- **优秀**：讲清自定义栈与浏览器 history 的同步难点（popstate）、KeepAlive 内存治理。
- **及格**：能说清页面栈 + KeepAlive + 动画的基本实现。
- **不足**：只能说"用了 KeepAlive 缓存页面"。

常见追问：1) KeepAlive 缓存过多怎么办？2) 从缓存页返回后怎么刷新数据？

项目五：一番赏 ONLINE (B5, 2 题)

【B5-01】【L2】Sentry + 自定义埋点数据体系

题目：你用 Sentry + 自定义埋点统计用户操作，为经验值/成就系统与后台分析提供数据，埋点是怎么设计的？怎么保证数据不丢不乱？

思考方向：埋点模型、上报时机与方式、异常监控与业务埋点的职责区分。

考察点：前端数据链路设计。

参考答案

- 职责区分：Sentry 负责**错误/异常/性能**监控 (JS 报错、接口失败、白屏、慢加载)，通过 sourcemap 还原堆栈；自定义埋点负责**业务行为** (进入页面、点击、下单、完成任务)。
- 埋点设计：统一埋点方法 (事件名 + 业务参数 + 用户标识 + 时间戳 + 页面路径)，集中定义事件枚举避免命名混乱；关键路径 (支付、任务完成) 加入唯一 ID 便于去重与归因。
- 上报可靠性：批量 + 延迟上报 (如队列攒够 N 条或定时 flush) 减少请求；页面卸载用 sendBeacon/keepalive 保证最后一批不丢；失败本地暂存重试。
- 性能：高频事件 (滚动、曝光) 用防抖/节流 + 采样，避免埋点本身拖垮页面 (项目里对高频埋点做了防抖节流)。
- 数据消费：经验值/成就规则基于事件流计算，后台做漏斗与留存分析。

评分要点

- **优秀：**讲清异常监控与业务埋点的分工、批量/beacon 上报保障、以及埋点自身的性能治理。
- **及格：**能说清理点要素与上报方式。
- **不足：**只会说“接了 Sentry”。

常见追问：1) 埋点数据怎么和“经验值系统”对账？2) 埋点采样后统计口径会不会失真？

【B5-02】【L2】访问速度提升 1.3s 与 blurify 差分、多端适配

题目：一番赏项目里“访问速度提升 1.3s”和“blurify 分辨率差分实现未登录/登录/VIP 区分”，这两件事具体怎么做、怎么量化？

思考方向：性能优化手段 + 资源分级加载策略。

考察点：性能量化、资源策略设计。

参考答案

- blurify 差分：对大量图片做模糊处理作为低清预览 (不同用户等级展示不同清晰度——未登录看模糊图、登录/VIP 看清晰图)，既控制资源版权/成本，又通过“先模糊占位再替换高清”改善首屏观感。实现上按用户身份决定加载哪一档资源，配合懒加载避免一次性请求大量高清图。
- 1.3s 提升来源：① 图片懒加载 (IntersectionObserver + 自定义指令) 避免首屏请求全部漫画图；② 埋点与高频函数防抖/节流；③ ECharts 按需引入 + Tree Shaking 减少体积；④ 多端适配 (安全区/Rem/Viewport、媒体查询) 减少无谓重排；⑤ 路由/资源分包。
- 量化：用 Performance API / Lighthouse 采集优化前后指标 (如 FCP/LCP、完全加载时间)，对比得出 1.3s 提升；说明测试环境与样本。
- 诚实口径：要明确 1.3s 是“某关键页面”还是“全站平均”，避免夸大。

评分要点

- **优秀：**讲清差分资源的用户分级策略与“模糊占位”体验价值，并说明性能数字的采集方式与口径。
- **及格：**能说清图片懒加载、按需引入等优化手段。
- **不足：**只说“优化了图片”，说不出量化方法。

常见追问：1) 模糊图转高清图怎么避免闪烁？2) 多端适配中遇到过什么具体兼容坑？

C. 场景 / 系统设计题 共 4 题

【C-01】【L3】设计企业级多租户 RAG 知识问答平台

题目：基于你做的企业问答 Agent，设计一个可服务多个部门/多租户、支持知识库版本管理与权限隔离的 RAG 平台，给出架构与关键决策。

思考方向：按"接入层 → 应用编排 → RAG 检索 → 存储 → 治理"分层，重点讲隔离、更新与评测。

考察点：RAG 系统工程化、多租户与可运维性。

参考答案

- **接入层：**网关统一鉴权 (JWT/OAuth2) + 限流 + SSE 流式；按租户/部门路由到独立配额。
- **应用编排：**意图识别 → 查询改写 (Rewrite/HyDE) → 混合召回 → RRF → Rerank → 生成 → 引用标注/拒答；低置信转人工。用状态机或 LangGraph/AgentScope 编排，会话上下文存 Redis。
- **检索与存储：**Milvus 向量库 + BM25 引擎；chunk 带 tenant/dept/role/密级 元数据，**pre-filter** 做权限隔离；小规模租户可 PGVector，超大租户可独立 collection。
- **知识治理：**离线 Pipeline (解析/清洗/语义分块/父子文档/向量化/入库) + 增量更新 (文档变更事件驱动)；**版本管理** (知识库按版本快照，支持灰度切换与回滚)；删除要同步清向量。
- **评测与运维：**评测集 + RAGAS 指标回归 (每次知识库/参数变更都跑)；线上监控 (召回率、Faithfulness、拒答率、转人工率、P95 延迟、token 成本)；Badcase 闭环归因。
- **成本：**语义缓存、小模型预筛、Rerank 候选数控制、按租户配额与计费。

评分要点

- **优秀：**pre-filter 隔离、版本管理与回滚、评测驱动迭代、成本配额——四点齐备。
- **及格：**架构分层完整，讲清权限隔离与更新机制。
- **不足：**只画"文档→向量库→检索→回答"的框图，无隔离/版本/评测/成本。

常见追问：1) 文档删除后向量怎么同步清？2) 知识库更新如何做灰度不翻车？

【C-02】【L3】设计支撑 1000+ 并发、首 token < 300ms 的 AI 客服系统

题目：设计一个支撑 1000+ 并发对话、首 token < 300ms 的 AI 客服系统，给出架构与关键决策。

思考方向：接入 → 会话 → Agent/RAG → 模型 → 人工，全链路谈并发、延迟与降级。

考察点：高并发系统设计、流式与容灾。

参考答案

- **接入层：**网关限流 (按用户/租户) + 负载均衡；SSE 长连接 (关闭 Nginx 缓冲、放大超时)，连接数与资源要评估上限。
- **会话层：**Redis 存会话上下文 (按 sessionId + TTL)，状态机管理 bot/pending_human/human；分布式会话保证多实例切换不丢。
- **Agent 编排：**意图识别 → RAG 检索 → 模型生成 → 敏感检测 → 转人工；高频 FAQ 走语义缓存直出 (0 模型调用，是压低首 token 的关键)。
- **模型层：**推理服务 (vLLM + Continuous Batching + PagedAttention) 提吞吐；多模型路由 (简单问题小模型/缓存，复杂问题大模型)；超时降级到规则或转人工。
- **流式实现：**生成任务与 HTTP 连接解耦，streamId + event id 支持断线重连续传；前端 SSE 渲染，首 token 目标 < 300ms (靠缓存命中 + 模型首字优化 + 就近部署)。
- **容灾与观测：**模型/向量库故障降级；全链路监控 (首 token、P95、连接数、断线率、解决率、转人工率、满意度)；成本控制 (缓存命中率、token 用量)。

评分要点

- **优秀：**讲清"生成与连接解耦 + 语义缓存压低首 token + 多模型路由降级 + 全链路观测"，并指出 SSE 长连接的资源瓶颈。
- **及格：**架构完整、有会话管理与降级思路。
- **不足：**只有框图，缺并发/延迟/降级的具体决策。

常见追问：1) 300ms 首 token 具体靠什么达成？2) 上下文超长如何截断又不丢关键信息？

【C-03】【L3】设计对话式数据分析（Text2SQL）助手

题目：设计一个"用自然语言查数据、出图表、可导出"的数据分析助手，覆盖多表、口径复杂、权限敏感的场景。

思考方向：Schema RAG + 口径知识 + SQL 校验 + 只读沙箱 + 可视化。

考察点：Text2SQL 系统工程化。

参考答案

- **理解层：**用户问题 → 意图/实体识别 → Schema 检索（只注入相关表，用 RAG 检索表与字段描述）→ 口径知识库（指标定义、计算公式）注入。
- **生成层：**LLM 生成 SQL（Few-shot 示例 + 多轮澄清）；复杂问题拆解为多步；结果不符预期时自动纠错重试（把错误信息回灌模型）。
- **安全层（重点）：**SQL 解析器做语法与语句类型校验（禁 DDL/DML）、表白名单、只读账号、强制 LIMIT 与超时、走只读从库、行级/字段级权限、敏感字段脱敏、全量审计。
- **呈现层：**结果表格 + 自动选图（趋势→折线、对比→柱状）+ 导出 Excel（流式导出防 OOM）；支持追问下钻。
- **评估：**构建"问题→标准 SQL→标准结果"的评测集，评估执行准确率（EX）与结果一致率；对失败案例分类归因（表选错/口径错/语法错）。

评分要点

- **优秀：**Schema 检索 + 口径知识 + 只读白名单/LIMIT/超时的安全闭环 + 评测集，四点齐备。
- **及格：**讲清 Schema 注入、生成 SQL、只读权限与基本校验。
- **不足：**直接"让模型写 SQL 执行"，无安全与评测。

常见追问：1) 模型生成的 SQL 正确率怎么提升？2) 多口径指标怎么保证一致？

【C-04】【L3】设计千万级文档的入库 Pipeline（含增量更新）

题目：要给知识库灌入千万级文档并持续增量更新，Pipeline 怎么设计？怎么处理解析失败、重复与更新？

思考方向：离线批处理 + 增量事件驱动，重点讲幂等、去重、可观测与成本。

考察点：数据工程、批处理流水线设计。

参考答案

- **分层 Pipeline：**接入（多源：文件/OA/代码库/数据库）→ 解析（PDF/Word/HTML/扫描件 OCR）→ 清洗（去噪、去页眉页脚）→ 分块（语义 + 父子）→ 向量化（批量 + 并发 + 重试）→ 入库（向量库 + 元数据）。每阶段可独立扩缩容。
- **异步与解耦：**用消息队列（Kafka/RabbitMQ）串联各阶段，生产者-消费者模型；大批量用分片 + 并发 worker，控制下游（向量库、Embedding 服务）压力。
- **幂等与去重：**文档指纹（内容 hash）作唯一键，重复不重复入库；更新时"先删旧 chunk 再写新 chunk"（或用版本号 + 逻辑删除），保证不残留过期知识。
- **失败处理：**单文档失败不阻塞整批，记录失败原因入死信/重试表，支持人工重试；解析质量差（扫描件）标记待人工处理。
- **增量更新：**变更源（定时扫描 / 事件订阅）→ 只处理变更文档；代码仓库可用 webhook 触发。
- **可观测与成本：**进度看板（已处理/失败/速率）、Embedding 调用量与成本统计、失败率告警；对超大文档做截断或分级处理控制成本。

评分要点

- **优秀：**讲清分阶段解耦、内容指纹幂等、"先删后写"的更新策略、失败隔离与增量触发。
- **及格：**能给出多阶段流水线 + 队列 + 重试的基本设计。
- **不足：**只说"写个脚本遍历文档入库"，无幂等/失败/增量设计。

常见追问：1) 千万级向量入库时向量库扛不住怎么办？2) 如何保证更新期间检索不出现"半新半旧"？

E. 非技术类问题 共 10 题

【E-01】【L1】自我介绍（前端 5 年 → AI Agent 方向）

题目：请你做个自我介绍。

思考方向：3 段式——我是谁/技术底色 → 我做了什么（重点 AI 项目）→ 我为什么投这个岗。

考察点：表达结构、与岗位的匹配度。

参考答案

- **开头 (15s)**：5 年开发经验，前 3 年深耕前端（Vue3/TS、组件库、工程化与性能优化），近 1 年转向 AI Agent 应用开发，主导过企业级智能问答 Agent。
- **主体 (60s, 重点讲 AI 项目)**：用 Java 生态（AgentScope/Spring AI）搭 RAG 问答 Agent，打通"意图识别→混合召回→RRF→Rerank→生成→工具调用"，把销售高频问题一次答准率做到约 85%，并做了 RAGAS 离线评测与人工接管兜底。
- **收尾 (20s)**：技术栈横跨前端 + Java 后端 + 微服务 + AI 应用，能独立完成从后端接口到前端交互的完整交付；希望在 AI Agent 方向长期深耕，所以投这个岗位。
- **要点：主动把"前端背景"转化为优势**（全栈交付能力、组件化思维、性能与工程化意识），不要让面试官自己去拼凑你的定位。

评分要点

- **优秀**：有清晰主线（全栈 + AI 方向），把 AI 项目讲成亮点，并主动弥合"前端转 AI"的疑虑。
- **及格**：按时间线讲清经历与技术栈。
- **不足**：流水账式背简历，重点落在早年前端而 AI 项目一带而过。

常见追问：1) 那你还做前端吗？2) 你说的 Agent 项目里最难的点是什么？

【E-02】【L2】为什么从前端转 AI Agent 方向

题目：你做了 3 年前端，为什么转 AI？是不是跟风？

考察点：职业动机的真实性与稳定性。

参考答案

- 动机要有"内因 + 外因"：内因是对大模型能力的兴趣、做过智能问答后感到"用 AI 解决实际业务问题"的价值感；外因是公司业务需要，我主动承接了 AI 问答项目，从 RAG 到 Agent 一步步做起来。
- 强调**不是抛弃前端而是能力叠加**：前端背景让我能做 AI 产品的交互与流式体验（SSE、渲染性能），全栈视角反而稀缺。
- 用行动证明：不是"喊着要转"，而是已经交付了项目、系统学过 LangChain/LangGraph、看了框架源码。
- 避免踩雷：不要说"前端没前途/太卷了/待遇低"。

评分要点

- **优秀**：动机真实（兴趣 + 业务驱动）+ 已用项目证明 + 强调能力叠加而非否定过去。
- **及格**：能说出合理的转型理由。
- **不足**：说"AI 火，工资高"，暴露跟风。

常见追问：1) 如果这个岗也用不到前端，你接受吗？2) 你觉得自己现在是前端还是 AI 工程师？

【E-03】【L2】离职原因

题目：为什么想离开现在的公司？（简历工作经历 2021.09-2023.07 / 2023.07-2024.08 / 2024.11-至今）

考察点：稳定性、职业成熟度（简历有几次跳动，必被细问）。

参考答案

- 原则：**向前看（追求什么）**而不是向后抱怨（逃离什么）。
- 可以说：现公司项目进入稳定维护期，AI 相关的迭代空间有限，希望进入以 AI 应用为主业的团队，把 Agent 方向做深；对贵司在 XX 方向的投入很认可。
- 对早期两次变动（1 年 / 1 年 3 个月），用客观事实解释并快速带过：如项目收尾/业务方向调整/个人方向明确，重点强调“每段都有交付与成长”，把落点放在“现在方向明确了，希望稳定深耕”。
- 红线：不要吐槽前领导/同事/公司管理，不要显得“一遇到问题就走”。

评分要点

- **优秀**：理由积极且与求职方向自洽，对短期经历有客观解释，展现稳定性诉求。
- **及格**：理由合理、态度平和。
- **不足**：抱怨前东家、或理由前后矛盾。

常见追问：1) 上一份只做了一年多，这次能保证做多久？2) 如果新公司也让你做前端，你怎么想？

【E-04】【L1】职业规划

题目：未来 3 年的职业规划是什么？

考察点：目标感、稳定性、与岗位的匹配度。

参考答案

- 结构化：**1 年**——吃透业务与技术栈，独立负责 AI 应用模块，把 RAG/Agent 的工程化能力打磨扎实（含评测、监控、成本）；**2-3 年**——成为团队里 AI 应用方向的技术骨干，能主导中大型系统设计、做技术选型与方案评审，并带少量新人。
- 方向要收敛：说“深耕 AI Agent / RAG 工程化”，而不是“什么都想学”或“想转管理”。
- 与公司绑定：希望在一个业务稳定、有真实 AI 落地场景的团队里成长，而不是频繁换赛道。

评分要点

- **优秀**：分阶段、方向收敛、与岗位/公司发展契合。
- **及格**：有明确方向且愿意深耕。
- **不足**：答“走一步看一步”或“想创业/想转管理”，与岗位预期不符。

常见追问：1) 如果 3 年内没有升职机会你怎么想？2) 你更想做业务还是做平台？

【E-05】【L2】最有挑战的项目与如何解决

题目：讲一个你遇到过的最有挑战的问题，你是怎么解决的？

考察点：问题拆解、韧性、复盘能力（用 STAR 结构）。

参考答案

- **S/T（背景）**：企业问答 Agent 上线后，销售反馈“经常答不准、答非所问”，但不知道问题出在哪。
- **A（行动）**：① 搭建离线评测集（抽样高频问题 + 人工标注标准答案）；② 用 RAGAS 指标把链路拆开定位，发现主要是召回问题（专有名词、型号漏召）；③ 引入 BM25 + 向量混合召回，加 RRF 融合与 Rerank 精排；④ 优化分块（语义分块 + 父子文档）；⑤ 建立 Badcase 归因机制持续迭代。
- **R（结果）**：高频问题一次答准率提升到约 85%，并沉淀了一套可复用的评测与迭代流程。
- 加分点：讲清“当时数据说不清哪里错”这一最难的阶段，以及“先建立度量再优化”的思路，而非直接讲技术方案。

评分要点

- **优秀**：STAR 完整，突出"先度量定位、再针对性解决"的方法论与量化结果。
- **及格**：能讲清背景、行动与结果。
- **不足**：只讲技术细节，没有难点与取舍，或结果无量化。

常见追问：1) 如果重做一次你会怎么改进？2) 团队里其他人在这个项目做了什么？

【E-06】【L2】技术选型分歧怎么处理

题目：如果你和同事在技术选型上（比如用 Dify 还是自研 Agent）有分歧，怎么办？

考察点：沟通协作、技术判断的理性程度。

参考答案

- 先把分歧"翻译成可验证的问题"：列出各自方案的评估维度（开发效率、可控性、运维成本、数据合规、长期演进、团队能力），而不是争论"哪个技术好"。
- 用**小成本验证**替代口头争论：做 POC（一周内跑通核心链路），拿数据说话；分歧大时用评分表加权决策。
- 尊重决策机制：技术负责人拍板后即使与自己意见不同也全力执行，并把风险提前记录（如"该方案的运维成本需关注"），而不是消极配合。
- 举例：项目里"平台快速验证 vs 代码自研落地"就是用"平台做 MVP、跑通后代码重写核心链路"折中的。

评分要点

- **优秀**：强调"可验证维度 + POC 数据决策 + 尊重最终决策"，有真实例子。
- **及格**：能说出理性沟通、以数据/需求为准。
- **不足**：回答"听领导的"或"坚持自己对"。

常见追问：1) 如果你的方案被否了但你确信它对，怎么办？2) 有没有说服过别人的经历？

【E-07】【L2】抗压能力与线上故障处理

题目：你怎么看加班/压力？线上出故障你会怎么做？

考察点：责任心、应急心态、复盘意识。

参考答案

- 态度：能接受项目关键期的高强度投入，同时更认同"用工程手段减少长期加班"（自动化、监控、规范）。
- 故障处理顺序：**先止血**（回滚/降级/限流/切流量，恢复可用性优先）→ **再定位**（看监控/日志/链路追踪）→ **后复盘**（写清根因、影响面、改进项与责任人），并沉淀为预案。
- 举例：如缓存变更导致的回源压力，用"先限流保护、再定位到刷新入口、最后加鉴权与分流"的顺序处理，体现"先保业务再找根因"。
- 心态：故障时不甩锅、主动同步进展，让团队和业务方知道"正在处理、何时恢复"。

评分要点

- **优秀**：给出"止血→定位→复盘"的清晰顺序，有具体经历，体现责任心与改进意识。
- **及格**：能说明不推诿、会复盘。
- **不足**：说"我不接受加班"或"出了故障看谁改的代码"。

常见追问：1) 你处理过的最严重故障是什么？2) 复盘时发现是自己的问题怎么写？

【E-08】【L2】跨岗位协作与需求冲突

题目：如果产品经理的需求你评估技术上行不通或代价太大，你怎么沟通？

考察点：沟通技巧、换位思考、方案意识。

参考答案

- 不直接说"做不了", 而是回到**业务目标**: 先问清需求背后要解决什么问题, 往往能找到等效替代方案 (成本低 80%)。
- 把技术代价翻译成 PM 能理解的语言: 工期、风险、对现有功能的影响、长期维护成本, 并给出 2-3 个带取舍的方案让其选择 (推荐方案 + 折中方案 + 完整方案)。
- 无法达成一致时升级决策: 拉上技术负责人一起评估, 而不是在群里僵持。
- 举例: 产品要"AI 回答 100% 准确", 无法承诺; 我改成本实现为"提高召回准确率 + 低置信转人工 + 明示不确定性", 既满足业务体验又技术上可交付。

评分要点

- **优秀**: 先对齐业务目标、给多方案 + 取舍、有真实沟通例子。
- **及格**: 能说明讲替代方案、摆事实沟通。
- **不足**: 回答"PM 说什么就做什么"或"直接怼回去"。

常见追问: 1) 需求临时加塞怎么排优先级? 2) 有没有说服 PM 放弃需求的经历?

【E-09】【L3】应对"技能太杂、深度不够"的质疑

题目: 你简历上前端、Java 后端、微服务、AI 都写, 是不是什么都懂一点、都不深?

考察点: 自我认知、技术深度自证 (这是本简历最可能被质疑的点)。

参考答案

- 承认广度, 但立刻把它转化为**可交付的深度证据**: 不是"学过", 而是"用它在真实项目里解决过什么量化问题"。
- 分层回答: ① 主攻方向 = AI Agent 应用开发 (RAG 全链路、Agent 编排, 能从评测、调参、兜底讲到线上指标); ② 支撑能力 = Java 后端与微服务 (订单查询 1s→300ms、缓存并发优化、限流降级); ③ 前端 = 工程化与性能 (首屏降低 30%-50%、组件库 20+)。
- 主动给出"深度证明"的例子: 如 RAG 混合召回的 RRF 公式与 k 值调优、按商品 ID 加锁 + 二次检查的实现细节、线程池并行汇总的取舍——用一个能讲透的技术点证明自己的深度。
- 姿态: 不辩解、不虚张, 坦诚"深度集中在 AI 与后端, 前端是熟练级", 比硬撑"都很深"更可信。

评分要点

- **优秀**: 用"主攻/支撑/熟练"三层定位 + 真实量化案例, 主动自证深度。
- **及格**: 能说清自己的主攻方向与各方向的掌握程度。
- **不足**: 强调"我什么都深", 被追问细节就露怯。

常见追问: 1) 挑一个你最有把握的技术点, 我深入问一下。2) 你最近在系统学习什么?

【E-10】【L2】期望薪资 (17k-20k) 与反问环节

题目: 你的期望薪资是多少? 你还想了解什么?

考察点: 薪资谈判技巧、求职意愿与主动性。

参考答案

- 报价策略: 先了解岗位薪资结构 (底薪 / 绩效 / 年终 / 补贴 / 五险一金基数 / 试用期折扣), 再给**区间**而非死数: "综合目前 5 年经验与 AI 方向能力, 期望 17k-20k, 具体看岗位职责与整体结构。"
- 锚定价值: 强调 AI Agent 应用开发的稀缺性 (Java + AI 双栈、能独立交付全链路), 而不是"我生活成本高所以要多"。
- 弹性表达: 如果岗位发展空间好, 薪资可在合理范围内协商, 展现意愿但不轻易自破底价。
- 反问 (体现主动性, 务必准备 2-3 个): ① 团队目前 AI 应用在业务中的定位与下一步规划? ② 这个岗位现在最急需解决的问题是什么? ③ 团队的技术栈与协作方式、以及对我前 3 个月的期待? 避免一上来就问加班和年假。

评分要点

- **优秀**：先了解结构再报区间、用能力锚定价值、反问有深度且体现对业务的思考。
- **及格**：给出合理区间与追问薪资结构。
- **不足**：报出远高于市场的数字却说不出依据，或答"都可以/按公司标准"，或反问答"没什么问的"。

常见追问：1) 如果我们只能给到 16k 你来吗？ 2) 手上还有其他 offer 吗？